

1.Introducción

Este proyecto consiste en el diseño e implementación de un portal Web para el Hospital Universitario San Cecilio dedicado a la agrupación, creación y modificación de entradas con información de distintas asociaciones médicas.

El documento que se presenta está dividido en siete capítulos, a los que se les añade un capítulo para la bibliografía y referencias adjuntadas en forma de anexo. En este mismo capítulo, se realiza una pequeña introducción al proyecto, haciendo una breve exposición del propósito del mismo. Además se explican de forma general, los objetivos y el alcance previsto.

En el segundo capítulo, el *Estudio de viabilidad*, se realiza un análisis del objeto del proyecto, haciendo hincapié en la situación de estudio y en los objetivos del mismo. Además, se especifican los recursos a utilizar tanto en el desarrollo del proyecto como en su futuro funcionamiento. En este capítulo también se incluye la planificación del desarrollo.

En el tercer capítulo se realiza en *Análisis del proyecto*, dónde se especifican los requerimientos funcionales y no funcionales del mismo. Además se incluyen restricciones de diseño que condicionen el desarrollo del proyecto.

En el cuarto capítulo se detalla el *Diseño* del proyecto. En primer lugar se describen los entornos de desarrollo y el sistema de almacenamiento seleccionado. También se detallan las diferentes capas que forman la aplicación, así como la arquitectura de la misma. Por último se describe el diseño para las interfaces de usuario.

En el quinto capítulo, se describe la *implementación*. Por tanto, en este capítulo se describe la codificación y el marcado de las diferentes capas de la aplicación, como resultado de los requisitos funcionales previstos anteriormente y el diseño realizado.

En el sexto se describen las *Pruebas* hechas una vez finalizada la implementación de la aplicación. Estas pruebas nos aseguran el correcto funcionamiento del proyecto.

En el séptimo capítulo, *Conclusión y resultados*, se explica la consecución de los objetivos. Además se proponen líneas de mejora del proyecto.

Por último se presenta la *Bibliografía*, donde se exponen las fuentes de información consultadas y el Anexo.

1.1. Objetivo y Alcance Previsto

En este documento se presenta el proyecto que consiste en el diseño e implementación de un portal Web para el Hospital Universitario San Cecilio.

La empresa solicitante desea un portal donde un usuario administrador, pueda dar de alta a distintas organizaciones asociadas a la empresa.

Además la empresa cuenta con una página web que trata de fomentar la colaboración creando redes de apoyo y propiciando el trabajo conjunto entre los profesionales del hospital y las entidades sin ánimo de lucro vinculadas a la salud. Por otro lado trata de dar visibilidad en mostrar y difundir la importante función que desarrollan las asociaciones de pacientes en áreas como el voluntariado, la prevención y la educación para la salud. Además, cuenta con una difusión periódica siendo así, que cada quince días, da a conocer el trabajo que realiza una asociación específica en conjunto con el hospital.

Por otro lado, la implantación del portal Web puede mejorar el dinamismo y facilitar a los administradores el trabajo, además de facilitar a los usuarios una vista más sencilla dinámica, por lo que, el portal vendrá complementado con una serie de funcionalidades para la difusión de información relacionada con la empresa.

El objetivo principal del proyecto es facilitar al usuario común el acceso a la información de distintas asociaciones sin ánimo de lucro, y brindar acceso a los contactos de las mismas a través de un portal web. Así mismo, busca brindar al administrador facilidad para crear, modificar, eliminar y filtrar asociaciones. De esta manera se mejora el servicio que ofrece la empresa en el momento de la solicitud del proyecto.

Por otro lado se desea tener un directorio con la información de las distintas asociaciones, categorías y etiquetas para el correcto funcionamiento de la página.

1.2. Estado del arte

El portal Web que se presenta con este proyecto, es básicamente un portal para la creación de entradas dinámicas con información de distintas asociaciones, las cuales incluyen apartados como nombre, siglas, descripción, logo, cartera de servicios, archivos multimedia, categorías, etiquetas etc. Existen diversos portales que ofrecen el mismo o parecido servicio a otras empresas. *Censo de Asociaciones en Salud*, *Directorio de Pacientes Sacyl*, *Directorio Salud Mental Andalucía*. A continuación vemos un pequeño análisis de los portales de estas empresas:

- [Censo de Asociaciones en Salud de la Junta de Andalucía](#): Es la red de apoyo oficial de la Junta de Andalucía. Su interfaz permite a los usuarios consultar todas las entidades legalmente inscritas, ofreciendo filtros por

provincia, por tipo de asociación o mediante un buscador directo por nombre.

- [Directorio de Pacientes Sacyl de Castilla y León](#): El portal de salud de Castilla y León mantiene un registro público dividido por categorías clínicas. En cada categoría se vuelca la información de contacto operativo como lo son teléfono, email, web y dirección física de las asociaciones que operan en su territorio.
- [Directorio de Salud Mental de Andalucía](#): Es un catálogo gestionado por la federación autonómica que lista asociaciones de salud mental. Su estructura de datos es casi idéntica a la tuya, mostrando el nombre de la entidad, dirección, teléfono, correo electrónico, página web y presidencia de forma estructurada.

2. Estudio de Viabilidad.

2.1. Introducción al estudio de viabilidad.

En el presente capítulo se realiza un análisis de diferentes aspectos del proyecto, a fin de determinar la viabilidad del proyecto.

En primer lugar se describe el objeto del proyecto, haciendo hincapié en la situación de estudio, proponiendo una mejora a esta, y en los objetivos del proyecto. Seguidamente se especifican los perfiles de usuario que harán uso de la aplicación resultante del proyecto. A continuación se especifican los objetivos que se pretenden conseguir con la aplicación resultante del proyecto. En el siguiente apartado se da una descripción del sistema a realizar, exponiendo de forma general las funcionalidades con las que contará la aplicación. En el siguiente se estudian los recursos y software necesarios para el desarrollo, implantación y funcionamiento del proyecto. Teniendo en cuenta estos recursos y el sistema a implantar, se describe la planificación del proyecto, ordenando y describiendo las tareas a realizar. Finalmente se exponen las conclusiones del presente estudio de viabilidad.

2.2. OBJETO

2.2.1. Situación actual.

Actualmente, la iniciativa cuenta con un portal web activo e impulsado por la Comisión de Participación Ciudadana del Hospital Universitario Clínico San Cecilio de Granada. Esta plataforma se utiliza para canalizar la colaboración entre los profesionales sanitarios y las entidades sin ánimo de lucro vinculadas a la salud, difundiendo de forma quincenal el trabajo conjunto en materia de voluntariado, prevención y educación sanitaria.

Además, a día de hoy, el proyecto se encuentra consolidado tras su primer año de funcionamiento, habiendo alcanzado la participación de un total de 26 asociaciones y fundaciones mediante campañas periódicas de visibilidad y el uso de contenido audiovisual.

2.2.2. Propuesta de mejora.

La mejora que se propone con este proyecto es realizar un portal web, En este se podrá consultar información de las Asociaciones de manera centralizada visual y

dinámica, Además se añadirán diversas funcionalidades para que los futuros espectadores puedan obtener información de estas, y los usuarios administradores puedan modificar y ampliar la información insertada.

2.2.3. Perfiles de usuario del proyecto

La aplicación a desarrollar está destinada a tener 2 tipos de perfiles de usuario:

- Perfil usuario no registrado. Podrá acceder cualquier persona que desee obtener información de las asociaciones así como de los servicios que ofrece.
- Perfil gestor de contenidos. Este usuario se encargará de actualizar los catálogos de productos y servicios de la empresa, de actualizaciones y de modelos de funcionamiento.

2.2.4. Objetivos

La implantación del portal Web que se presenta en este proyecto tiene como objetivo principal permitir a los usuarios de la web la consulta de la información y primera toma de contacto con distintas asociaciones relacionadas con el hospital a través de internet, de forma dinámica organizada por categorías y etiquetas, además de un sistema de búsqueda de los nombres y siglas de las asociaciones. Esto mejoraría de manera considerable los servicios que ofrece la web hasta ahora. Este servicio debe ser cómodo y de fácil manejo para ambos usuarios. Esto permitirá dinamizar el intercambio de información con los usuarios y las asociaciones y, así mismo, se alcanza un objetivo no menos importante que es el de incrementar el alcance de ayudas y servicios a pacientes que lo necesiten.

Por otro lado, de momento, la web no contará con una base de datos, sino que la información de las asociaciones se almacenará en un directorio en la raíz del proyecto.

Otro objetivo a largo plazo es aplicar la infraestructura del proyecto a un servidor real, pues en este momento no contamos con acceso a este.

2.3. Descripción del sistema

2.3.1. Descripción.

La aplicación que se presentará al final del proyecto será una web. Esta tendrá una parte estática, donde se expondrá contenidos informativos de la empresa, y otra parte dinámica donde los usuarios encargados de gestionar el contenido se encargará de rellenar un formulario, el cual al ser validado almacenará la información en un archivo .json, el cual será volcado en la página estática

Funcionalidades de la página

La web que desarrollamos tendrá las siguientes funcionalidades básicas:

- Sistema de Filtrado. Las entradas contarán con una categoría y varias etiquetas, de manera que en la página principal mediante un desplegable en el caso de las categorías, y un buscador de texto libre en las etiquetas, el usuario común pueda filtrar el contenido de esta. Además en un futuro este sistema deberá de contar con un sistema de búsqueda en texto libre el cual permite buscar coincidencias en el nombre o siglas de las asociaciones.
- Sistema de creación de entradas dinámicas. Mediante un formulario, el usuario gestor de contenido, rellenando el mismo podrá crear las diferentes entradas, asignando categorías, o etiquetas, y en el caso de que no existan crearlas desde el mismo formulario, de manera automática en el caso de las etiquetas o con una opción en el desplegable en el caso de las categorías.
- Sistema de autenticación. Mediante el cifrado de un token gestionado actualmente en el backend, en este caso un servidor de python al tener que simular el backend de un entorno real. La autenticación se fundamenta en el estándar JSON Web Token, al validar las credenciales el backend emite un token efímero firmado criptográficamente que el frontend almacena y adjunta obligatoriamente en la cabecera del HTTP de cada petición de escritura. Esto permite al servidor verificar la integridad de la sesión de forma que antes de procesar cualquier acción, rechaza instantáneamente cualquier solicitud ilegítima o acceso directo a la API mediante respuestas HTTP 401.

2.4. RECURSOS

Para la realización del proyecto, se necesitará una serie de recursos software, hardware y humanos que se detallan a continuación.

2.4.1. Recursos humanos.

Para la realización de este proyecto, en el apartado de recursos humanos, tan solo se contará con la aportación del alumnado de prácticas, que deberá realizar todas

las tareas necesarias para la finalización de los objetivos propuestos por el cliente en el periodo de sus estancia en la empresa de prácticas.

2.4.2. Recursos hardware.

Este proyecto se trata de la implementación de un portal Web, por lo que se necesita de un servidor para el hospedaje del portal. Además es necesario un ordenador personal para el acceso a la Web.

2.4.2.1. Recursos mínimos del cliente

Para el acceso a la Web el cliente deberá hacerlo a través de su ordenador personal. Este deberá contar con una conexión a internet igual o superior a 56 Kbps, además de un navegador web.

2.4.2.2. Recursos mínimos del servidor.

Para el hospedaje del portal, se hará uso de un servidor Web. Los recursos mínimos de este servidor, dependen del volumen de visitas que la empresa tenga previstas para la web. Otro factor que condiciona estos recursos es el tamaño de los archivos que almacenan la información de las asociaciones.

2.4.3. Recursos software.

Tanto para el diseño y el desarrollo del proyecto, como para su futuro funcionamiento, se requiere una serie de conjuntos de recursos software que se describen a continuación. El uso de estos recursos está justificado de manera más extensa en el apartado 4.1.1 Selección del entorno de desarrollo.

2.4.3.1. PC cliente

Es necesario que el cliente cuente con cualquier sistema operativo que contenga una interfaz gráfica, ya sea basados en Windows, Mac, o distribuciones de Linux. Además deberá acceder a la Web mediante Microsoft edge, Brave, Firefox, Chrome, etc, ya que el proyecto está pensado para ser visualizado en navegadores web.

2.4.3.2. Servidor

El servidor que se utilice para hospedar la web en un futuro deberá soportar protocolos HTTPS con sus respectivos certificados y DNS, y en el caso de que los archivos de almacenamiento migren a futuro a una base de datos, deberá

soportar bases de datos relacionales como MySQL o MariaDB. El sistema operativo a usar en el servidor es irrelevante ya que la página está pensada para funcionar tanto en sistemas windows como en sistemas Linux, aunque se recomienda usar un sistema operativo basado en el kernel de linux como lo es Debian Server debido a que cuenta con procesos más ligeros y eficientes debido a su huella de memoria considerablemente menos a la de Windows Server. Además de ser más estable ya que está pensado para largos periodos sin reiniciarse a diferencia de Windows Server que debe reiniciarse periódicamente debido a parches y actualizaciones. Relacionado a cargas de trabajo como servidores web también cuentan con mejor rendimiento en un 33% y con el 75% menos de consumo de memoria

2.4.3.3. Entornos de programación

Para la implementación del proyecto se han seleccionado diferentes entornos de lenguaje de marcado y de programación:

- HTML (HyperText Markup Language) y CSS (Cascading Style Sheets) para el diseño de la aplicación.
- Vanilla JavaScript para la lógica de la validación de formularios y creación de tarjetas o entradas
- Python3 para la lógica del backend, simulando un servidor al no tener acceso a ninguno real
- Markdown: para documentación del código en un archivo README.md

Además se recurrirá al siguiente software para el apoyo en diferentes necesidades:

- Visual Paradigm UML, para el diseño de diagramas de casos de uso y diagramas de secuencia
- Google Office, Documentos y Hojas de cálculo para la documentación

2.5. Planificación del Proyecto

2.5.1. Planificación inicial.

El proyecto empezará el 15 de mayo de 2026 y se prevé que el alumnado acabe su estancia en la empresa el 2 de junio de 2026 por lo que no se tiene en cuenta la finalización del proyecto a la hora de la entrega de este documento. Durante este periodo se le dedicará al proyecto una media de 6 horas diarias durante 5 días a la semana. El uso de estos días se hará de manera flexible, de manera que no influya en el rendimiento de otras actividades.

2.5.2. Organización del proyecto

El proyecto se desarrollará siguiendo las siguientes etapas:

1. Capturar requerimientos de usuario: Esta etapa se realizará mediante sprints siguiendo la metodología ... con el cliente con el fin de establecer pequeños objetivos del proyecto y toda la información necesaria para la realización del mismo.
2. Establecer requisitos funcionales y no funcionales del sistema: una vez obtenida toda la información del cliente, en el periodo del sprint, se redactarán los requisitos del proyecto con el fin de identificar todas las funcionalidades y restricciones.
3. Proponer una solución: tras especificar los requisitos del proyecto y estudiarlos se propone una solución.
4. Estudiar alternativas: En el caso que existan alternativas a la solución propuesta se comparan estas para escoger la que mejor se adapte al problema.
5. Buscar y definir los recursos software y hardware para desarrollar el proyecto: en esta parte del proyecto se seleccionarán los recursos que sean necesarios para la realización de la aplicación en los plazos deseados.
6. Planificar las tareas a realizar que requiere el proyecto: en esta etapa se realizará toda la planificación del proyecto, indicando las tareas y distribuyendo los recursos que se dispongan a dichas tareas.
7. Realizar el proyecto: una vez realizada la planificación se procederá a la implementación del portal, así como todos los procesos para que este funcione.

2.5.3. Modelo de desarrollo

El modelo de desarrollo que se ha elegido para la ejecución del proyecto es el modelo Scrum. Se ha optado por este modelo ya que únicamente se dispone de feedback constante por parte del cliente, permitiendo redireccionar el software en desarrollo a las opiniones y consejos del mismo, con el fin de que el producto final sea en totalidad o casi totalidad las preferencias del cliente

2.6. CONCLUSIONES

Una vez realizado el estudio de viabilidad, se puede extraer las siguientes conclusiones:

- El proyecto presentará beneficios a los usuarios de la web ya que facilitará el acceso a la información de las asociaciones.

- Permitirá crear de manera cómoda y sencilla las entradas y la información de las mismas al gestor de información

3. ANÁLISIS DEL PROYECTO

3.1. INTRODUCCIÓN.

En el presente capítulo se describen los requerimientos funcionales de los diferentes perfiles de usuario que tiene el proyecto.

Como complemento a la descripción de los requerimientos funcionales se incluyen una serie de diagramas de casos de uso para cada perfil de usuario, además de diagramas de secuencia para las funcionalidades más destacadas.

A continuación se detallan los requerimientos no funcionales del proyecto, teniendo en cuenta las restricciones asociadas al diseño así como los objetivos del diseño.

3.2. REQUERIMIENTOS FUNCIONALES.

Los requerimientos funcionales hacen referencia al comportamiento de la aplicación, con todas aquellas funcionalidades que la integran. Estos requerimientos funcionales están representados en los diagramas de uso de cada tipo de usuario presente en el proyecto. Además se presentan los diagramas de secuencia de cada uno de los requisitos funcionales.

3.2.1. Visión general.

El proyecto que se presenta en este documento es la implementación de un portal web para el Hospital Universitario San Cecilio el cual busca centralizar y agrupar todas las asociaciones que colaboran con el Hospital en una misma página web.

La Web que se obtendrá como resultado de este proyecto tiene como objeto principal el facilitar el acceso a la información de distintas asociaciones y a la creación, modificación y borrado de las entradas de las mismas.

El portal necesitará de un usuario gestor de contenidos, que se encargue de actualizar los datos de la web. Esta gestión consistirá básicamente en actualizar los ficheros que almacenan los datos de las asociaciones, estos datos serán brindados por cada asociación y serán actualizados mediante un formulario desde el panel de administración.

Por otro lado, el portal servirá como ya se ha mencionado antes, para proporcionar información a futuros usuarios no registrados, esta información estará dividida en

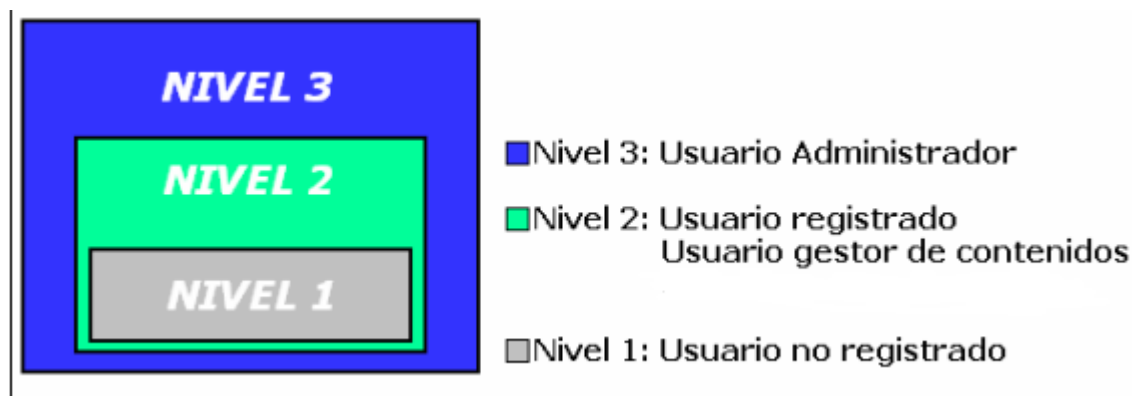
varias partes, La parte de la asociación, y el apartado multimedia dentro de las mismas, donde se mostrará contenido específico de cada una.

Finalmente, se abre la puerta a la existencia de un usuario administrador, que pueda crear más usuarios gestores de información pero que no se ha implantado de momento debido al corto tiempo de estadía del alumnado.

3.2.2. Requisitos funcionales del usuario

Como se explica en el apartado anterior, la aplicación dispondrá inicialmente de 1 usuario gestor de información el cual estará registrado, con sus requisitos funcionales específicos y el resto de usuarios serán no registrados.

Así mismo, existirá un nivel de privilegios: el primer nivel formado por el usuario no registrado, un segundo nivel en el que estarán incluidos el usuario gestor de contenidos y finalmente si fuese necesario, un usuario administrador. Los usuarios de cada nivel tendrán asignadas sus funcionalidades pero heredarán las de los niveles inferiores, de este modo, los usuarios del tercer nivel podrán acceder a sus funcionalidades y además a las funcionalidades de segundo y primer nivel de privilegios.



A continuación se presentan los requisitos de funcionamiento específicos de cada perfil de usuario definido en el proyecto.

3.2.2.1. Usuario no registrado

El usuario no registrado o visitante ocasional, dispondrá de los siguientes requisitos funcionales:

- Acceso al contenido público.

Los usuarios no registrados podrán entrar al portal Web para acceder a todos aquellos módulos que no necesiten identificación a fin de consultar información referente a las asociaciones además de ponerse en contacto con estas.

- Autenticarse

Todos aquellos usuarios que hayan obtenido credenciales de inicio de sesión de manera legítima, deberán hacer una autenticación (o log-in), para poder acceder al contenido protegido.

3.2.2.2. Usuario Registrado

El usuario registrado heredará las funcionalidades del usuario no registrado, y además dispondrá de los siguientes requisitos funcionales:

- Creación de entradas: Una vez se haya autenticado con éxito el usuario, tendrá acceso a un nuevo módulo en el menú de navegación, el cual le permitirá, mediante a un sencillo formulario, crear las entradas de las asociaciones de manera dinámica y automática
- Modificación y borrado: En el caso de querer modificar o eliminar una existente, el usuario podrá notar como en la página principal al entrar a una entrada hay dos nuevas opciones, donde la primera, permitirá mediante un nuevo formulario modificar los datos necesarios, y en el caso de la segunda opción eliminar la entrada completamente

3.2.2.3. Usuario administrador

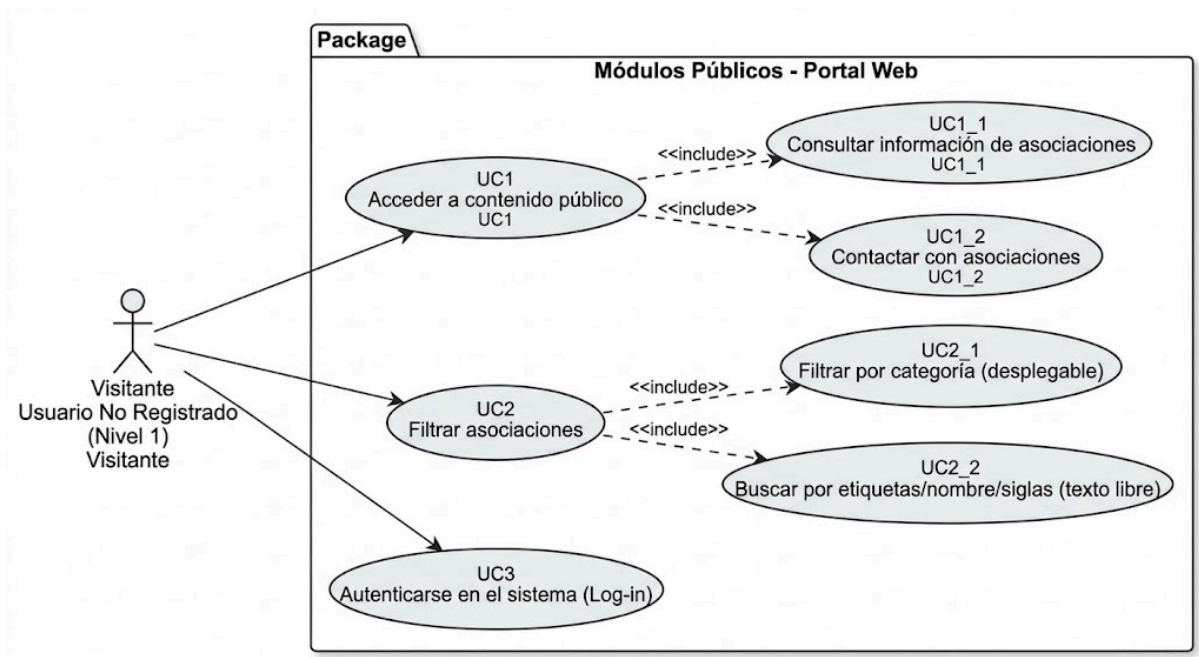
El usuario administrador heredará las funcionalidades disponibles para los niveles inferiores de usuario de la aplicación y además dispondrá de los siguientes requisitos funcionales:

- Gestión de perfiles: A cada perfil de usuario, como ya se ha explicado, le corresponden unas determinadas funcionalidades. El administrador, en el caso que sea necesario, podrá cambiar estas credenciales o añadir nuevas, de manera que se le pueden añadir funcionalidades a cada perfil de usuario.

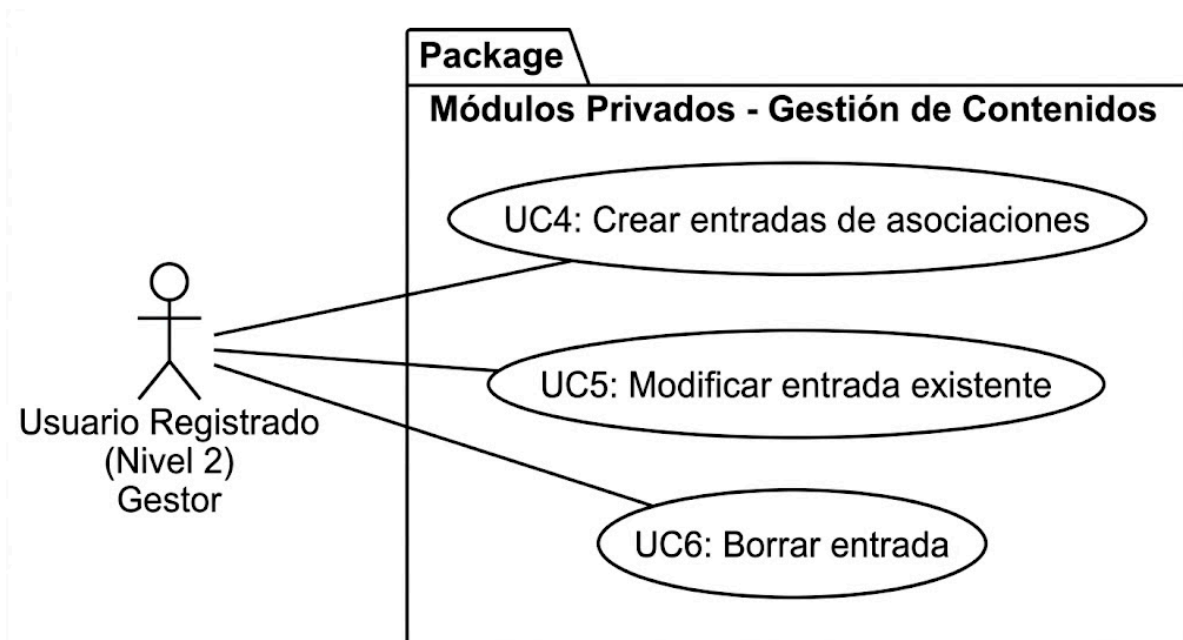
3.2.3. Diagramas de casos de uso

A continuación se muestran los casos de usos de cada perfil de usuario según se han especificado en el apartado 3.1.2 Requisitos funcionales de usuarios

3.2.3.1. Diagrama de casos de uso para usuarios no registrados.



3.2.3.2. Diagrama de casos de uso para usuarios registrados.



3.2.4. Diagramas de secuencias.

3.2.4.1. Diagramas de secuencia para usuario no registrado.

En las siguientes figuras se muestran los diagramas de secuencias correspondientes a las funcionalidades del usuario no registrado.

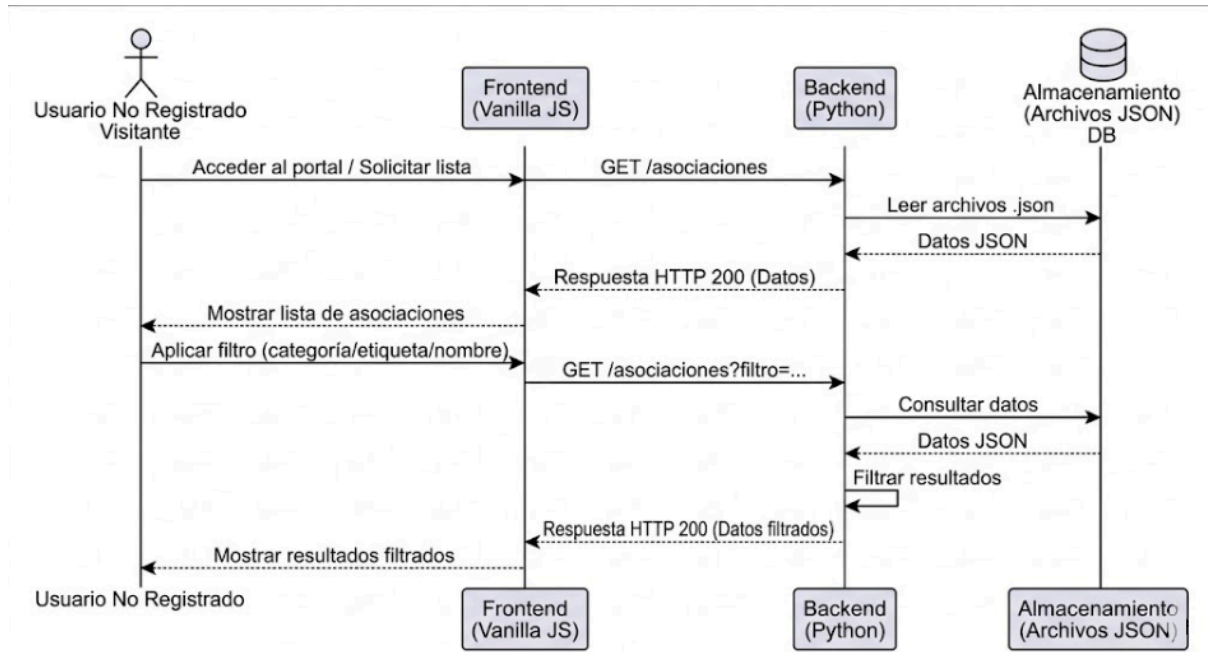
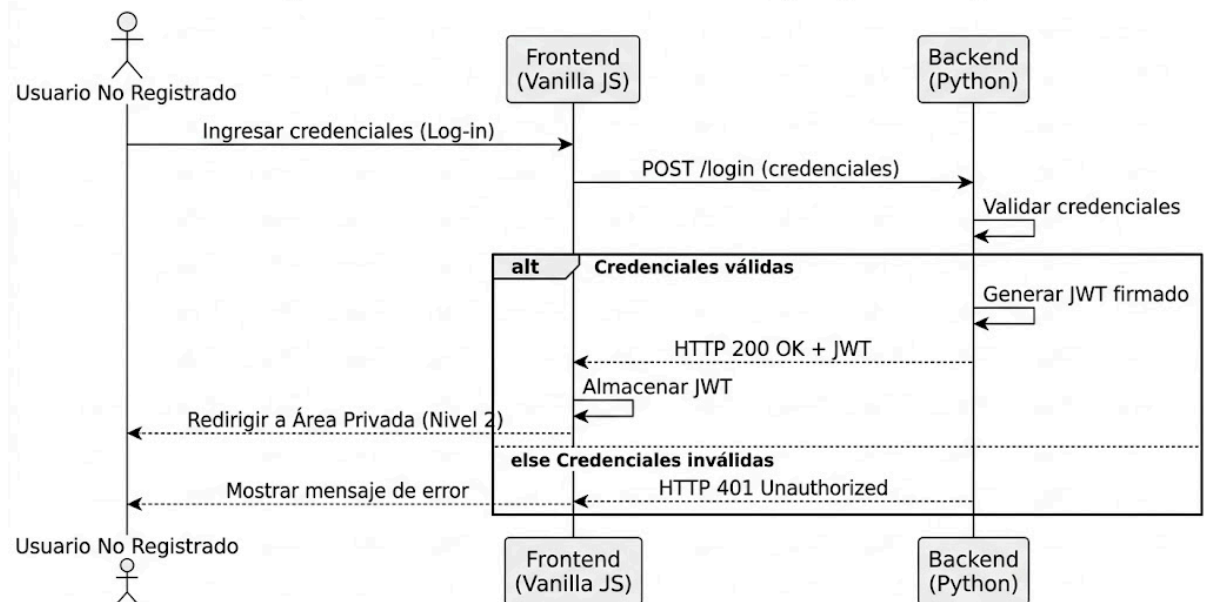


Diagrama de Secuencia: Proceso de Log-in (Visitante)



3.2.4.2. Diagramas de secuencias para el usuario gestor de contenidos

Diagrama de Secuencia: Proceso de Creación de Entrada (Gestor)

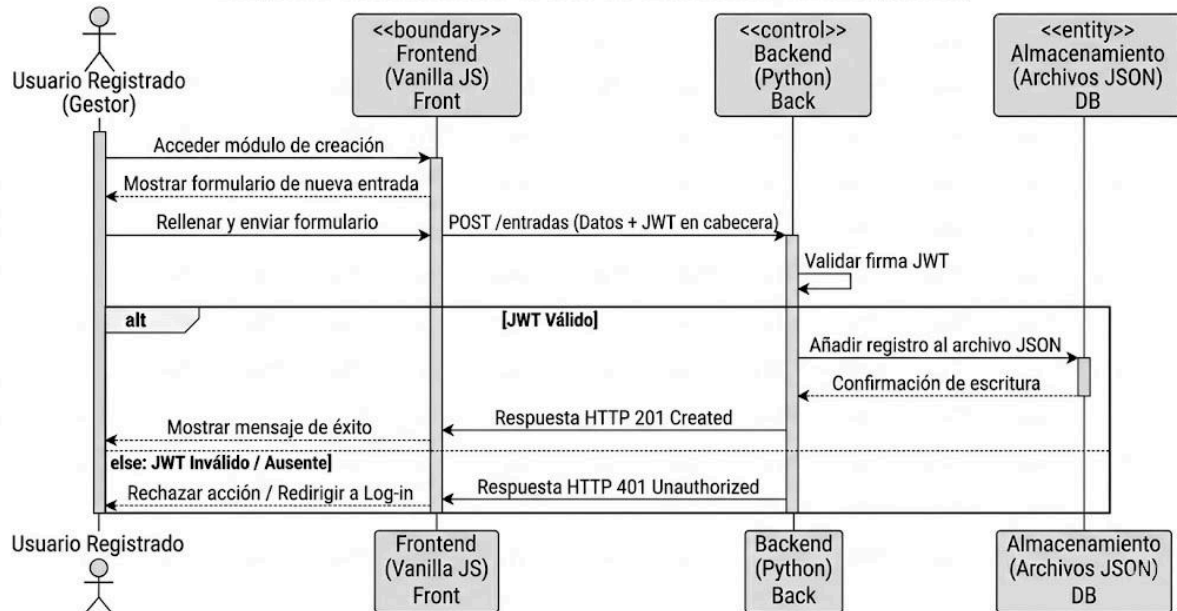
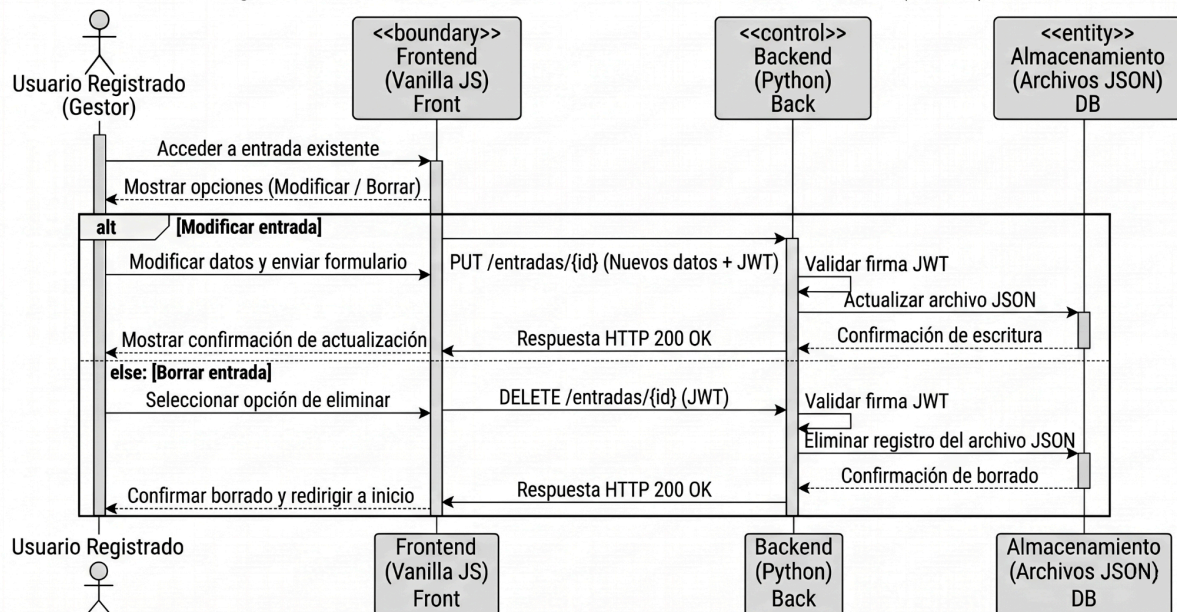


Diagrama de Secuencia: Proceso de Gestión de Entrada Existente (Gestor)



3.2.5. Menús de navegación.

Al portal acceden diferentes perfiles de usuario, cada uno con funcionalidades específicas, Por tanto es necesario aplicar un sistema de menús de navegación. Este sistema se encargará de permitir a cada usuario únicamente el acceso a las funcionalidades que le sean permitidas en función al perfil que le pertenece.

Mediante un menú estático se accederá a aquellas funcionalidades que sean comunes para todo tipo de usuario que acceda al portal.

3.3. REQUERIMIENTOS NO FUNCIONALES

Los requerimientos no funcionales hacen referencia a las restricciones impuestas por el cliente y el propio desarrollador y restricciones impuestas por la dependencia de la aplicación con otras aplicaciones o software ya desarrollado o diseñado a continuación se detallan las restricciones no funcionales que se presentan en el proyecto.

3.3.1. RESTRICCIONES DE DISEÑO

Dependencia de otras funcionalidades.

En este presente proyecto, se pretende realizar una funcionalidad que permita al usuario actualizar las entradas que portan la información de las distintas asociaciones. Por lo tanto la aplicación deberá poder realizar una conexión con el servidor del Backend, que en este caso se trata de manera temporal un script de python, el cual se encargará de cifrar el tráfico de los jwt cifrado, para acceder a los datos se deberá de acceder a unos archivos en texto plano (.json) para cargarlos en el frontend.

Seguridad

El portal web está pensado para que sea utilizado por diferentes perfiles de usuarios, cada uno con acceso a diferentes funciones. Por tanto es necesario implementar un sistema de control de accesos.

Este sistema procurará básicamente permitir el acceso a las diferentes funcionalidades únicamente a los usuarios que están autorizados. Por lo tanto, cada usuario tendrá una contraseña que solo conocerá el, y que utilizará para autenticarse.

Una vez hecha esta identificación, en el menú de navegación se mostrarán todas aquellas funcionalidades a las que el usuario puede acceder debido a sus características.

Si un usuario intenta acceder a una funcionalidad a la que no está autorizado, se le impedirá el acceso.

Prevención de errores en los formularios.

La mayoría de las funcionalidades con las que contará la web, requieren rellenar formularios para la introducción de datos de la web. Aunque la mayoría de estas funcionalidades estarán disponibles para perfiles de usuario con conocimientos específicos de su área de trabajo, es necesario comprobar que se rellenan todos los campos obligatorios de los formularios y en caso que sea necesario, comprobar que estos datos sean coherentes, para no comprometer la integridad de los ficheros de datos.

3.3.2 OBJETIVOS DE DISEÑO.

Programación estructurada

El portal web que se obtendrá como resultado de la implementación del proyecto, es susceptible a sufrir mejoras o ampliaciones de las funcionalidades. Por ello, es conveniente que toda la programación se realice de forma modular y estructurada, además de utilizar todos los recursos de programación que permitan la comprensión del código fuente.

Por otro lado, la utilización de CSS puede facilitar los cambios futuros en el diseño del portal, por lo que su aplicación es altamente recomendable.

Claridad.

Es importante que toda la información que se presente en el portal se muestre de forma clara y coherente, es decir, utilizar un diseño que no dificulte la comprensión. Además el diseño debe ser formal ya que se trata de una web de un Hospital

Accesibilidad y fácil manejo.

En el portal existirán múltiples funcionalidades, y cada perfil de usuario podrá acceder a varias de estas. Es por eso que se debe facilitar al usuario el acceso a estas. Esto se conseguirá con un sistema de menús, con un menú específico a cada perfil de usuario, que permita el acceso a las funcionalidades de su perfil de usuario.

4. DISEÑO

4.1. Introducción.

En este capítulo se analizará la solución propuesta a los requerimientos que se enumeran en el capítulo 3. Análisis del proyecto. Por tanto, todas las decisiones que se han tomado, y que están presentes en este capítulo, condicionan la resolución final del proyecto, inclusive, las posibles ampliaciones o mejoras de este.

En primer lugar se analizan los entornos de desarrollo que se emplearán en la implantación de los requisitos funcionales del proyecto, describiendo los motivos por los cuales se han seleccionado.

A continuación se describe la configuración de la plataforma donde se hará servicio a la aplicación.

En el siguiente apartado se enumeran y describen las diferentes capas de la aplicación (entorno de usuario, motor de la aplicación y capa de datos).

En este mismo capítulo también se incluye una descripción del diseño de las diferentes interfaces de usuario de la aplicación.

Por último se describe la estructura de archivos y la política de back-up's a implementar en el proyecto.

4.1.1. Selección del entorno de desarrollo.

Para la implementación de la aplicación web se utilizará un único entorno configurado para cubrir las necesidades que implica el proyecto. Por lo que usaremos Visual Studio Code con plugins específicos que nos permita desarrollar lo máximo posible las funciones de la página.

En primer lugar dispondremos del lenguaje de marcado HTML, basado en el uso de etiquetas. Este lenguaje nos permitirá mostrar información al usuario de manera sencilla y de una forma atractiva. Además permite la interactividad del usuario mediante formularios con los que obtener información. Por otro lado, su uso es sencillo y permite al desarrollador explotar todas las posibilidades con una curva de aprendizaje corta. A todo esto, cabe añadir, que su utilización es, a día de hoy, la más extendida y por tanto la compatibilidad del lenguaje con los diferentes navegadores Web que se utilizarán para el acceso a la aplicación es total.

Como complemento al lenguaje HTML, utilizaremos las hojas de estilo CSS. Crearemos una única hoja que defina diversas etiquetas, que pueden ser usadas por el HTML, y que definan el diseño del portal. Este entorno nos permite crear un diseño para la aplicación Web que cumpla los requisitos de apariencia y accesibilidad indicados por el cliente. Además permite al desarrollador crear un código claro y estructurado, de manera que el proyecto pueda ser modificado o mejorado de forma sencilla. Por otro lado, su uso nos permite crear diversas hojas de estilo para modificar el diseño a futuro.

Otro lenguaje que hemos elegido para la implementación del proyecto es el lenguaje JavaScript. Esto nos permitirá crear funciones que se requieran en el proyecto.

Su uso, dentro del desarrollo, se basará en la comprobación de datos introducidos por el usuario, o funciones de validación.

Por otra parte, para el guardado de información usaremos el formato JSON. Éste nos permitirá almacenar los datos de las distintas asociaciones directamente en un directorio en la raíz del proyecto, supliendo la ausencia de una base de datos en la etapa actual. Su uso, dentro del desarrollo de este proyecto, se basará en la persistencia de la información estructurada en archivos de texto plano. De esta manera, los datos introducidos y validados a través de los formularios del panel de administración se guardarán en dichos archivos, para posteriormente ser leídos y volcados en la parte estática de la página web.

Finalmente, para la implementación del backend se utilizará el lenguaje de programación Python en su versión 3. Su uso dentro del proyecto se fundamenta en la necesidad de simular un servidor, debido a la falta de acceso a un entorno real durante esta fase de desarrollo. En primer lugar, este script se encargará de gestionar el sistema de autenticación de los usuarios gestores de contenido. Al validar las credenciales de inicio de sesión, el servidor emitirá un token efímero firmado criptográficamente, basado en el estándar JSON Web Token (JWT). Además, el backend verificará la integridad de este token, que el frontend debe adjuntar obligatoriamente en la cabecera de cada petición de escritura. Esto permite al servidor rechazar instantáneamente cualquier solicitud ilegítima o acceso directo a la API mediante respuestas HTTP 401. Por otro lado, Python actuará como controlador principal para el procesamiento de las peticiones HTTP. Esto permitirá ejecutar las operaciones para añadir, actualizar o eliminar registros, interactuando de forma directa con los archivos JSON destinados al almacenamiento de datos.

4.2. Configuración de la plataforma

Una vez analizados los entornos de desarrollo que se utilizarán en el proyecto, debemos concretar la configuración de la plataforma donde se dará servicio a la aplicación.

En primer lugar, la aplicación WEB deberá estar hospedada en un servidor ASGI de alto rendimiento **Uvicorn**, bajo el framework **FastAPI**, de acceso remoto a través de Internet (soportando protocolos HTTPS con certificados SSL/TLS y DNS correspondientes). Este servidor es multiplataforma, pero aunque podría desplegarse en un servidor con cualquier sistema operativo, y su procedimiento fuese el mismo, se recomienda su despliegue sobre un sistema operativo basado en el kernel de Linux, específicamente **Debian Server**, debido a que ofrece una estabilidad superior frente a reinicios y optimiza los recursos de la máquina, presentando un consumo de memoria un 75% menor y un rendimiento un 33% superior frente a otros entornos como lo es Windows Server. La elección de este moderno servidor de ejecución asíncrona es debida a la necesidad de gestionar peticiones concurrentes y API de alto rendimiento con latencias extremadamente bajas.

Además, el servidor deberá dar soporte nativo al lenguaje de programación **Python 3.9 o superior** y deberá tener configurados y activados los módulos y librerías declarados en el archivo *requirements.txt*, incluyendo **PyJWT**, para la firma criptográfica y validación de tokens efímeros JSON Web Token, **Pydantic**, para el tipado y validación robusta de esquemas de datos previniendo corrupción en el almacenamiento, y **python-multipart**, para el procesamiento seguro de formularios de autenticación administrativa en formato URL-encoded. Asimismo, el servidor dará soporte para persistencia de datos en texto plano mediante archivos estructurados en formato **JSON**, (*asociaciones.json*, *categorias.json*, *etiquetas.json*, *solicitudes.json* y *config.json*), garantizando un acceso a disco directo y eficiente. Además, la plataforma debe permitir la escritura y almacenamiento en disco para la carga de logos guardados físicamente en *html/img/logos/* y la lectura temporal en memoria para la importación masiva de datos en formato CSV.

4.3 Capas de la aplicación

La aplicación a desarrollar estará constituida por tres capas diferenciadas.

4.3.1 Entorno de usuario

El entorno de usuario es donde se nos muestra la información e incluso se le solicita información al usuario. Como se explica en el apartado 4.1.1 Selección del entorno de desarrollo, esta capa se implementará utilizando lenguaje HTML y el uso de CSS.

En un archivo de estilo (*style_Form.css*) definiremos diversas variables en la raíz (*:root*), reglas y etiquetas que luego utilizaremos en los archivos con lenguaje HTML. Principalmente definiremos una paleta de colores corporativos (con el verde institucional *#008361* y el menta de interacción *#00b090* sobre fondos de contraste grises claros y blancos), y estructuramos las páginas utilizando

módulos flexibles (Flexbox) y cuadrículas (CSS Grid). Como complemento definiremos el estilo de la cabecera, el pie de página, ventanas modales emergentes y elementos de formulario (como bordes redondeados y transiciones suaves al pasar el cursor).

Una vez definidos todos estos elementos podremos aplicarlos al diseño del portal. En primer lugar situaremos el cuerpo de la página (``body`` y ``main``). Éste estará centrado en relación al área del navegador, limitando su ancho máximo a 900px para facilitar la lectura y configurado para empujar siempre el pie de página hacia el final de la pantalla (sticky footer). A este cuerpo de página le sobrepondremos los otros elementos principales del diseño:

- **La cabecera:** estará situada en la parte superior del cuerpo de la página ocupando todo su ancho. En esta cabecera se mostrará el logotipo de la asociación junto con un menú de navegación alineado a la derecha, todo sobre un fondo con el color verde principal del proyecto. Los enlaces administrativos en este menú ("Entradas" y "Cerrar Sesión") se mostrarán u ocultarán de manera condicional dependiendo de la presencia del token del administrador. Asimismo, incluye un enlace visible para todos los usuarios hacia la interfaz de solicitudes ("Solicitar Inclusión").
- **El módulo de inicio de sesión:** estará contenido en una caja de ancho reducido, centrada vertical y horizontalmente en la pantalla de acceso. Este módulo agrupará el formulario de acceso, los botones de acción con bordes curvados tipo píldora, y los mensajes de error dinámicos para gestionar el ingreso de los usuarios de forma segura.
- **El formulario de Solicitud de Inclusión (*contacto.html*):** es una interfaz estructurada en tres pasos dinámicos donde los representantes de asociaciones externas pueden registrar sus datos. Integra validación en tiempo real en los campos del cliente, una sección de geolocalización completa, País, Comunidad, Provincia y Municipio, una casilla obligatoria de consentimiento RGPD, protección antispam invisible Honeypot y un mensaje de éxito con código de referencia una vez completado el envío.
- **El contenido principal:** estará situado en el contenedor central de la página. En éste mostraremos las asociaciones estructuradas mediante un sistema de cuadrícula responsiva, donde cada elemento será una "tarjeta" interactiva que se eleva visualmente mediante transiciones de transformación en 3D al pasar el cursor por encima, facilitando la interacción del usuario y dándole un aspecto moderno y visualmente agradable.
- **El panel modal:** se desplegará sobreponiéndose a toda la pantalla, oscureciendo el fondo con una capa translúcida para centrar la atención del usuario. Ocupará una posición central y mostrará información ampliada del catálogo, logo en alta definición, descripción de actividades, lista de contactos y cartera de servicios detallada, e incluirá botones específicos de

administración para los gestores autenticados, además de un botón en la esquina superior derecha para cerrarlo y volver al listado general.

- **El pie de página:** estará situado en la parte inferior del cuerpo. En éste mostraremos información adicional estructurada en tres columnas de cuadrícula: la primera columna con el logotipo secundario e información física de ubicación del hospital, la segunda columna con enlaces corporativos y avisos legales, y la tercera columna con el contacto directo del directorio, utilizando colores de fondo oscuros para generar un fuerte contraste visual con el cuerpo de la página.

Para aplicar este diseño implementaremos en HTML el siguiente pseudocódigo:

```
INICIO HTML5
HEAD
  METADATOS (charset, viewport)
  VÍNCULO CSS (style_Form.css)
FIN HEAD
BODY
  HEADER
    IMG (Logotipo de la empresa)
    NAV
      UL (Menú de navegación: Inicio, Asociaciones, Panel)
    FIN NAV
  FIN HEADER
  MAIN (Contenedor principal centrado, max-width: 900px)
    SECTION id="autenticacion"
      DIV class="login-container"
        FORM
          INPUT type="text" (Usuario)
          INPUT type="password" (Contraseña)
          BUTTON type="submit" (Entrar)
        FIN FORM
      DIV class="error-msg" (Mensaje de error interactivo)
    FIN DIV
  FIN SECTION
  SECTION id="contenido-galeria"
    DIV class="container-tarjetas" (Display: grid)
      ARTICLE class="tarjeta"
        IMG (Logo asociación)
        H3 (Nombre asociación)
        DIV class="insignia-filtro" (Siglas)
        DIV (Contenedor de etiquetas múltiples)
        BUTTON (Ver Detalles / Modal)
      FIN ARTICLE
    FIN DIV
  FIN DIV
```

```

FIN SECTION
DIV id="modal-detalle" class="modal-activo/modal-oculto" (Overlay)
  DIV class="modal-contenido"
    BUTTON (Cerrar modal - ×)
    DIV class="modal-cabecera"
      IMG (Logo asociación)
      H2 (Nombre asociación)
      H4 (Siglas)
    FIN DIV
    P (Descripción y categorías)
    DIV (Contactos de la asociación)
    DIV class="caja-texto" (Cartera de servicios)
    DIV id="contenedor-borrado" (Acciones de administrador)
      BUTTON (Modificar)
      BUTTON (Eliminar)
    FIN DIV
  FIN DIV
FIN DIV
FIN DIV
FIN MAIN
FOOTER
  DIV class="footer-contenido" (Columnas grid)
    DIV (Columna 1: Datos y logo corporativo del hospital)
    DIV (Columna 2: Enlaces legales y privacidad)
    DIV (Columna 3: Información de contacto del directorio)
  FIN DIV
  DIV class="footer-bottom"
    P (Copyright)
  FIN DIV
FIN FOOTER
VÍNCULO JS
FIN BODY
FIN HTML5

```

4.3.2. Motor de la aplicación

El motor de la aplicación será el encargado de relacionar el entorno de usuario con la capa de datos. Esta capa elegirá y generará los contenidos a mostrar por el entorno de usuario que, en la totalidad de los casos, estarán almacenados en archivos locales en formato JSON. Así mismo, el motor de la aplicación, también recibirá información del entorno de usuario para que sea tratada y validada antes de persistir en la capa de datos.

Para implementar esta interacción entre las capas se recurrirá a una adaptación práctica del patrón de arquitectura de software **Modelo-Vista-Controlador (MVC)**, adaptado a las necesidades específicas del proyecto:

1. **Vista:** Constituida por el frontend interactivo en los navegadores clientes, compuesto por archivos HTML estáticos, hojas de estilos CSS y lógica de scripting en el lado del cliente escrito en **Vanilla JavaScript**. El usuario interactúa con la interfaz mediante clics, envíos de formularios o filtros.
2. **Controlador:** Compuesto por el backend estructurado en **FastAPI** que expone las rutas RESTful seguras (*GET, POST, PUT, DELETE*). Recibe las peticiones HTTP del frontend, valida la sesión mediante tokens JSON Web Token (JWT) con expiración temporal en la cabecera de la solicitud, intercepta y restringe peticiones ilegítimas arrojando códigos *401 Unauthorized*, e inyecta middleware con cabeceras HTTP seguras para mitigar ataques.
3. **Modelo:** Definido por los esquemas de validación de datos atómicos escritos bajo **Pydantic** en Python (*Asociacion, Categoria, Etiqueta, ConfigWeb, SolicitudAsociacion, RepresentanteSolicitud*), que definen el contrato exacto de campos para las entidades del catálogo, y la lectura/escritura de los ficheros físicos en texto plano JSON en el sistema de archivos del servidor.

El lenguaje escogido para implementar toda esta interacción en el servidor es **Python 3.9 o superior**, ejecutado a través del servidor asíncrono Uvicorn. Su compatibilidad con el entorno web es absoluta gracias al protocolo ASGI, permitiendo recibir y responder peticiones JSON con validaciones automáticas sumamente eficientes y con un sistema robusto de detección de errores.

Funcionalidades especiales

En el presente proyecto, destacan las siguientes funcionalidades que otorgan al mismo un carácter seguro y avanzado:

- **Gestión Unitario de Catálogo:** Permite añadir, *POST*, modificar *PUT* o eliminar *DELETE* asociaciones, categorías y etiquetas de forma individual basándose en su ID único. Esto evita la necesidad de descargar y subir la base de datos completa desde el cliente, previniendo graves problemas de concurrencia y sobrecarga de red.
- **Validación de Esquema con Pydantic:** El motor de la aplicación valida cada objeto recibido contra su modelo Pydantic correspondientes. Si algún campo es malformado o faltan datos requeridos, el backend detiene la petición e informa al frontend arrojando un código *400 Bad Request* o *422 Unprocessable Content*, garantizando la inmunidad del almacenamiento contra corrupción de datos.
- **Gestión de Solicitudes de Inclusión:** Sistema público e independiente para recolectar solicitudes de asociaciones externas bajo los modelos *SolicitudAsociacion* y *RepresentanteSolicitud*, almacenándose en un histórico dedicado en el estado *pendiente* hasta su revisión. El administrador puede cambiar su estado, *pendiente, revisada, aprobada, rechazada* o eliminarlas.

- **Carga de Imágenes Directa:** Endpoint seguro */api/upload-logo* que valida extensiones *.jpg, .jpeg, .png, .gif, .webp, .svg*, controla el peso (máximo 5MB) y genera nombres seguros UUID evitando colisiones físicas.
- **Importador Masivo Inteligente:** Endpoint */api/importar* que analiza en lote ficheros CSV con codificaciones duales, autodetecta delimitadores y realiza inserciones en cascada registrando categorías y hashtags automáticamente sin duplicidades.
- **Configuración en Tiempo Real:** Endpoint */api/config* bajo el modelo *ConfigWeb* para establecer el fondo del portal dinámicamente.

Uso de herramientas de apoyo

Para la implementación y administración del proyecto se ha decidido utilizar herramientas de apoyo que ayuden a simplificar las pruebas y asegurar el rendimiento:

- **Interactive Docs (FastAPI Swagger):** El motor expone de manera automática una interfaz gráfica interactiva de pruebas, */docs* y */redoc*, bajo la especificación OpenAPI, permitiendo validar y depurar todos los endpoints de forma visual.
- **Herramientas de Desarrollador del Navegador:** Utilizadas para inspeccionar en tiempo real las respuestas de red, las cabeceras de seguridad inyectadas por el servidor, el rendimiento de las operaciones asíncronas y el almacenamiento del JWT en el *localStorage*.

4.3.3. Capa de datos

En la capa de datos guardaremos toda la información estructurada que necesitemos (asociaciones médicas, categorías de filtrado y etiquetas del directorio público). Esta información la almacenamos en archivos locales de datos en texto plano diseñados específicamente para la aplicación. El sistema de almacenamiento utilizado son ficheros estructurados en formato **JSON**.

En esta capa, recibiremos información para la inserción, modificación, eliminación o consulta de datos procedentes del motor de la aplicación (Controlador de FastAPI). Toda esta interacción se realizará de forma atómica mediante operaciones de entrada/salida de archivos de Python (*with open()*), parseando los datos estructurados en formato nativo de listas y diccionarios y persistiendo los cambios con indentación formateada para mayor legibilidad y mantenimiento del directorio de datos.

4.4. ESTRUCTURA DE LA BASE DE DATOS

La base de datos diseñada para el presente proyecto estará estructurada en archivos JSON locales diseñados en función de las necesidades del proyecto. Cada

colección contendrá esquemas de datos bien definidos, donde almacenaremos la información deseada. Además, existirán relaciones lógicas mediante identificadores entre las diferentes colecciones, para facilitar el filtrado del catálogo y para realizar búsquedas en el lado del cliente de forma sumamente rápida.

4.4.1. Definición de Colecciones JSON

A continuación se detalla la especificación de campos, tipos de datos y restricciones para cada una de las colecciones JSON que componen nuestra base de datos local:

1. Colección Asociaciones (*asociaciones.json*)

Colección principal donde almacenaremos la información referente a las asociaciones médicas del hospital. Cuenta con los siguientes campos:

- **id**: identificador único asignado a cada asociación que nos asegura la integridad de los datos. De tipo entero, generado automáticamente en el servidor.
- **nombre_asociacion**: nombre completo y formal de la asociación médica. De tipo cadena de caracteres (String), campo requerido.
- **Siglas**: abreviación o acrónimo representativo de la entidad (ej. "AECC"). De tipo cadena de caracteres (String), campo requerido.
- **Logo**: enlace URL absoluto o ruta de imagen local del logotipo de la asociación. De tipo cadena de caracteres (String), opcional (si se deja vacío, la interfaz le inyectará de forma automática una imagen de salvaguarda segura).
- **descripción**: bloque informativo sobre el propósito, historia y actividades principales que realiza la asociación. De tipo cadena de caracteres larga (String), requerido.
- **categoria**: identificador único de relación. De tipo cadena de caracteres (String). Debe corresponder con un *id* existente en la colección *categorias.json*.
- **etiquetas**: listado de identificadores de relación. De tipo arreglo de cadenas de caracteres (*List[str]*). Cada elemento de la lista debe corresponder con un *id* existente en la colección *etiquetas.json*.
- **contactos**: listado de elementos estructurados de contacto. De tipo arreglo de objetos compuestos (*List[Contacto]*), campo requerido. Cada objeto contiene:
 - **tipo**: tipo de método de comunicación. Cadena restringida a los valores: X, Instagram, Facebook o TikTok u otros contactos no contemplados por la página.
 - **valor**: dirección, número o enlace web del método de contacto. De tipo cadena de caracteres String.

- **cartera_servicios**: lista en texto plano de las prestaciones, terapias, ayudas y servicios que ofrece la asociación a los pacientes y sus familias. De tipo cadena de caracteres de texto enriquecido/multilínea (String), requerido.
- **videos**: listado de enlaces a recursos audiovisuales de plataformas como YouTube.
- **ubicacion**: datos de localización física de la asociación. Contiene:
 - **pais**: país de localización (opcional).
 - **comunidad**: comunidad autónoma o región de origen (opcional).
 - **provincia**: provincia física. (opcional).
 - **municipio**: municipio o ciudad (opcional).

2. Colección Categorías (*categorias.json*)

Colección que define las clasificaciones clínicas en las que se agrupan las asociaciones médicas. Cuenta con los siguientes campos:

- **id**: identificador único de la categoría. De tipo cadena de caracteres con prefijo *cat_* y timestamp único, requerido.
- **nombre**: etiqueta legible de la clasificación médica (ej. "Cáncer", "Salud Mental", "Enfermedades Raras"). De tipo cadena de caracteres (String), requerido.

3. Colección Etiquetas (*etiquetas.json*)

Colección de descriptores o palabras clave en formato hashtag para realizar filtros rápidos. Cuenta con los siguientes campos:

- **id**: identificador único de la etiqueta. De tipo cadena de caracteres con prefijo *eti_* y timestamp único, requerido.
- **nombre**: término libre formateado obligatoriamente con el carácter # al inicio (ej. "#AECC"). De tipo cadena de caracteres, requerido.

4. Colección Solicitudes (*solicitudes.json`*)

Colección que almacena las solicitudes de inclusión enviadas por representantes de asociaciones externas para su posterior revisión por el administrador. Cuenta con los siguientes campos:

- **id**: identificador único con prefijo *sol_* seguido de un timestamp en milisegundos, requerido.
- **nombre_asociacion**: nombre completo y formal de la asociación propuesta, requerido.
- **siglas**: acrónimo representativo de la entidad, requerido.
- **descripcion**: misión y actividades de la asociación, requerido.

- **categoría_sugerida:** nombre o identificador de la categoría donde encaja la asociación, opcional.
- **web:** URL del portal o redes sociales de la entidad, opcional.
- **ubicacion:** objeto compuesto de la ubicación, opcional. Contiene:
 - **país:** país de origen, opcional.
 - **comunidad:** comunidad autónoma o región, opcional.
 - **provincia:** provincia física, opcional.
 - **municipio:** ciudad o término municipal, opcional.
- **mensaje:** texto libre con comentarios, motivación o notas complementarias, opcional.
- **representante:** datos del contacto físico que solicita el registro, requerido. Contiene:
 - **nombre:** nombre completo y apellidos del solicitante, requerido.
 - **cargo:** puesto o responsabilidad dentro de la entidad, requerido.
 - **email:** dirección de correo electrónico institucional de contacto, requerido.
 - **telefono:** número de teléfono de contacto, opcional.
- **fecha:** fecha y hora de recepción del formulario en formato ISO 8601 UTC (ej. "2026-05-25T13:00:00Z"), requerido.
- **estado:** estado de tramitación. Cadena restringida a los valores: *pendiente, revisada, aprobada o rechazada*.

5. Colección Configuración (`config.json`)

Colección de parámetros globales de la aplicación web y personalización de la interfaz. Cuenta con los siguientes campos:

- **fondo:** identificador o valor del estilo/color visual de fondo configurado para el catálogo, opcional.

4.5. ARQUITECTURA DE LA APLICACIÓN

Como ya se ha explicado en el apartado 4.3 Capas de la aplicación, disponemos de tres capas diferenciadas: el entorno de usuario, el motor de la aplicación y la capa de datos. Para gestionar la interacción entre estas tres capas utilizaremos el patrón de desarrollo de software **Modelo-Vista-Controlador**, con algunas modificaciones para adecuarlo a las necesidades específicas del proyecto actual basado en peticiones asíncronas desacopladas.

Este patrón de desarrollo de software está definido por un flujo similar al siguiente, donde la vista es nuestro entorno de usuario, el controlador es el motor de la aplicación y el modelo es la capa de datos:

1. **Interacción (Vista → Controlador):** El usuario interactúa con la interfaz de usuario en el navegador (Vista) mediante algún elemento interactivo (haciendo clic en el botón de ver detalles de una asociación, aplicando un filtro de categoría, o enviando el formulario administrativo de creación/edición).
2. **Solicitud Asíncrona (Fetch API):** El script en Vanilla JavaScript intercepta la acción del usuario, extrae los valores de los campos de la interfaz y realiza una petición asíncrona mediante la función *fetch()* hacia el controlador de FastAPI. En las peticiones de escritura, se inyecta obligatoriamente el JSON Web Token (JWT) en la cabecera *Authorization* de la petición HTTP.
3. **Procesamiento y Validación (Controlador):** El controlador (*server.py*) recibe la notificación de la acción solicitada. Si se trata de un endpoint de escritura, verifica la integridad y vigencia del JWT arrojando un código *401 Unauthorized* si es inválido. Si es válido, utiliza el objeto *Body(...)* para capturar el cuerpo de la petición y valida su esquema de datos contra los modelos Pydantic definidos, deteniendo el flujo e informando al frontend si encuentra datos corruptos.
4. **Acceso al Almacenamiento (Controlador → Modelo):** El controlador accede a la colección de datos física en disco, realizando la acción correspondiente (añadir, modificar, o eliminar el registro atómicamente).
5. **Respuesta y Renderizado Seguro (Controlador → Vista):** El controlador devuelve una respuesta estructurada en formato JSON al navegador. El JavaScript del cliente la procesa, aplica validaciones y filtros adicionales en local, y actualiza de forma reactiva y asíncrona el árbol DOM del navegador utilizando propiedades inmunes a inyecciones como *textContent* y *innerText*, finalizando el ciclo sin necesidad de recargar la página completa.

4.5.1. Flujos de Acción en la Arquitectura

En el presente proyecto, se pueden diferenciar básicamente dos tipos de funcionalidades según la interacción con el usuario: aquellas en las que simplemente se muestra información en el catálogo público, y aquellas en las que el administrador introduce información para actuar sobre la persistencia de datos del portal. Por tanto, definiremos dos flujos de acción para estas funcionalidades:

1. Flujo para funcionalidades de consulta

Es el flujo simplificado mediante el cual los usuarios visitantes (no registrados) navegan por el catálogo público y realizan filtros. Los pasos que realiza el sistema son:

1. **Solicitud:** El navegador del usuario inicia una petición asíncrona al cargar la página (*GET /api/{recurso}*) o al aplicar un filtro en los selectores.
2. **Consulta en disco:** El motor de FastAPI procesa la ruta, lee el archivo JSON correspondiente en el sistema de archivos y retorna el listado JSON.
3. **Filtrado interactivo:** El script de JavaScript del cliente (*index.js*) recibe los datos crudos, realiza las comparaciones de texto y de ids del buscador reactivo en memoria local, y renderiza la lista final actualizando los nodos del DOM de forma sumamente rápida.

2. Flujo para funcionalidades de gestión

Es el flujo robusto mediante el cual los administradores autenticados insertan, modifican o eliminan registros en el portal web. Los pasos que realiza el sistema son:

1. **Formulario y Entrada:** Mostramos el formulario en la interfaz administrativa para introducir o modificar la información (ej. añadir contactos, seleccionar etiquetas o definir una nueva categoría).
2. **Carga de Recursos Multimedia y Documentos:** En caso de adjuntar un logotipo de la asociación, el administrador puede subir el archivo de imagen directamente. El script de JavaScript realiza una petición *POST* al endpoint protegido */api/upload-logo* enviando el archivo en un cuerpo multipart. El controlador de FastAPI valida la extensión, permitiendo únicamente *.jpg*, *.jpeg*, *.png*, *.gif*, *.webp*, *.svg*, limita el peso a un máximo de 5MB, genera un nombre único UUID para mitigar colisiones y guarda el archivo en el sistema de archivos del servidor devolviendo su ruta pública para ser vinculada al registro.
3. **Importación Masiva (CSV):** Para la carga inicial o actualizaciones en lote de las asociaciones, el administrador puede subir un archivo CSV. El backend procesa la petición en el endpoint */api/importar* validando el token admin, decodifica el archivo, autodetecta el delimitador ```,`` o ``;`` e importa en cascada las asociaciones, autogenerando categorías y etiquetas nuevas que no existían de forma totalmente automática en disco.
4. **Validación del lado del Cliente:** El script de JavaScript *crearEntrada.js* recopila los datos y valida en primera instancia que los campos obligatorios cumplan con los formatos.
5. **Peticiones Secuenciales:** Si el administrador ha creado elementos nuevos en cascada, como una categoría inline o una etiqueta de texto libre que no existía, el script realiza peticiones *POST* previas al servidor para registrarlas y asegurar sus respectivos identificadores.

6. **Acción y Validación en el Servidor:** El script emite la petición principal adjuntando el JWT. El controlador de FastAPI valida el token, verifica que la firma coincida, aplica el validador de esquemas Pydantic y persiste de forma segura los cambios en el archivo JSON atómicamente.

7. **Notificación de éxito:** El servidor retorna al estado de éxito. El script de JavaScript recibe el JSON, alerta al administrador con una confirmación clara y le redirige de inmediato al listado actualizado del catálogo principal de asociaciones.

3. Flujo para solicitudes de inclusión

Representa el proceso por el cual entidades externas soliciten su alta en el catálogo y los administradores gestionan su aprobación o rechazo:

1. **Envío Público:** Un representante de una asociación accede al formulario público de solicitud, *contacto.html*, completa sus datos, los datos de la asociación, con geolocalización de país, comunidad, provincia y municipio, y acepta el tratamiento de datos de carácter personal bajo el RGPD. Tras superar una protección anti-spam, se emite una petición asíncrona *POST* al endpoint */api/solicitudes* sin requerir credenciales de acceso.

2. **Almacenamiento en Espera:** El controlador de FastAPI valida el formato de entrada mediante el esquema Pydantic *SolicitudAsociacion*, inyecta un identificador único con prefijo *sol_*, registra la fecha y hora UTC exacta y fija el estado por defecto a *"pendiente"*, guardando la solicitud en el archivo *solicitudes.json*.

3. **Auditoría y Gestión:** El administrador inicia sesión y accede al panel de administración de solicitudes. El cliente JavaScript consulta mediante *GET /api/solicitudes*, con token JWT en cabecera) y despliega el listado. El administrador puede:

- **Actualizar el Estado:** Cambiar el estado de la solicitud mediante *PUT /api/solicitudes/{solicitud_id}* enviando un JSON con el nuevo valor, *pendiente, revisada, aprobada, rechazada*.
- **Eliminar el Registro:** Borrar la solicitud mediante *DELETE /api/solicitudes/{solicitud_id}* una vez tramitada o descartada para limpiar el histórico del sistema.

4.6. INTERFAZ

Para el presente proyecto se ha creado un diseño formal y sumamente minimalista que será aplicado, de forma general, a todas las interfaces de la aplicación, como se detalla en el apartado 4.3.1 Entorno de usuario. En este diseño, la visualización de la información y los elementos interactivos variarán de forma reactiva en el lado del cliente dependiendo del nivel de privilegio en el que se encuentre el usuario: el menú dinámico superior, el catálogo interactivo y los botones de acción administrativa.

4.6.1. Definición de Interfaces para los Diferentes Niveles de Privilegio

Para el acceso a las diferentes funcionalidades del portal web, se ha diseñado un comportamiento adaptativo controlado mediante Vanilla JavaScript. Este variará de acuerdo con los privilegios del usuario activo:

- **Nivel 1 (Usuario Visitante - Sin Sesión):** Nivel público. Puede acceder a la página principal, buscar por nombre o siglas, filtrar asociaciones usando los selectores desplegables de categoría, etiqueta y ubicación, País, Comunidad, Provincia, y hacer clic en las tarjetas de la galería para redirigir al usuario a una página de detalle dedicada, *detalle.html*, con información completa de la asociación, su localización geográfica, métodos de contacto seguros y galería multimedia.
- **Nivel 2 (Administrador / Gestor - Con Sesión Iniciada):** Hereda los permisos del visitante y agrega herramientas avanzadas de manipulación del catálogo. En la cabecera del portal se inyectan dinámicamente opciones administrativas, "Entradas" y "Cerrar Sesión". En la vista de detalle dedicada, *detalle.html*, se despliega el panel inferior de control con las opciones de **Modificar**, que redirige al formulario *crearEntrada.html* y **Eliminar**, que borra la entrada tras confirmación de alerta. Asimismo, tiene acceso exclusivo a la creación, edición, subida de logos, importación en lote de archivos CSV y configuración estética del fondo de la web.

4.6.2. Especificación Detallada de Vistas

A continuación se describen las interfaces diseñadas para cumplir con las funcionalidades más importantes del portal:

1. Interfaz de Catálogo Público (*index.html*)

Es el panel general de la aplicación. Muestra a todos los usuarios el directorio de asociaciones con su respectivo buscador y filtros reactivos.

- **Buscador Reactivo:** Un campo de texto libre inyectado de forma dinámica al inicio de la página que filtra tarjetas en tiempo real al escribir palabras clave que coincidan con el nombre o siglas de la asociación (ej. "AECC").
- **Selectores de Filtros:** Selectores desplegables cargados asincrónicamente desde el servidor que filtran las tarjetas por Categoría Médica y Etiquetas.
- **Filtros de Ubicación Avanzados:** Tres selectores dinámicos adicionales cargados dinámicamente desde el conjunto de datos de localización real de las asociaciones para filtrar por Provincia, Comunidad Autónoma/Región y País.
- **Galería de Tarjetas:** Cuadrícula responsiva que organiza las asociaciones en tarjetas individuales. Cada tarjeta renderiza de forma segura el logo, nombre, siglas, insignias (badges) de múltiples etiquetas y los datos

resumidos de ubicación geográfica. Al pasar el cursor por encima, la tarjeta se eleva ligeramente con un sombreado suave. El clic en cualquier tarjeta redirige a la ficha técnica de la asociación *detalle.html?id=...*

- **Aplicación Estética de Fondo Global:** Carga e inyecta dinámicamente una imagen de fondo definida en la configuración general del portal *config.json*, aplicando un gradiente translúcido para garantizar un contraste visual.

2. Interfaz de Ficha Detallada de la Asociación (*detalle.html*)

Página independiente de detalles profundos que despliega la información técnica y de contacto completa de una asociación mediante su ID en la URL:

- **Cabecera de Ficha:** Muestra el logotipo de la asociación en alta resolución, nombre formal completo, siglas destacadas, categoría de filiación, insignias de etiquetas activas y su localización geográfica completa.
- **Descripción de Actividades:** Bloque principal de texto donde se detalla el área de acción y labor comunitaria de la entidad.
- **Información de Contacto Validada:** Lista interactiva de contactos inyectada dinámicamente que formatea cada método de comunicación de manera segura.
- **Cartera de Servicios:** Cuadro estético de texto que detalla las prestaciones asistenciales o de asesoramiento para pacientes y familiares.
- **Galería Multimedia de Vídeos:** Sección oculta por defecto que se activa si la asociación cuenta con enlaces multimedia, renderizando una cuadrícula con reproductores de vídeo integrados (como youtube) de forma responsiva.

Panel de Acciones del Administrador: Panel inferior visible únicamente si hay una sesión activa, con botones integrados de "Modificar Asociación" y "Eliminar Asociación".

3. Interfaz de Autenticación (*login.html*)

Interfaz segura que permite al administrador validar sus credenciales y obtener su token de sesión JWT.

- **Contenedor Centrado:** Caja minimalista sobre fondo gris que agrupa el logo oficial verde del Hospital Clínico San Cecilio, campos de texto para Usuario y Contraseña, y un botón de ingreso de color verde corporativo con bordes curvados tipo píldora.
- **Gestión de Errores Reactiva:** Contiene un bloque de alerta oculto por defecto (*#error-msg*). Si el administrador introduce credenciales incorrectas, el script intercepta el código de error *401* del servidor y despliega instantáneamente una advertencia en color rojo indicando que verifique sus datos.

- **Redirección de Salvaguarda:** Si un usuario intenta acceder a esta vista cuando ya tiene una sesión activa, token presente en *localStorage*, el navegador lo redirige automáticamente a *index.html* sin mostrar el formulario.

4. Interfaz de Formulario Administrativo (*crearEntrada.html*)

Formulario interactivo y dinámico de uso exclusivo para administradores, protegido en el cliente mediante redirecciones automáticas si no se detecta sesión. Funciona de manera híbrida según el contexto de la ruta:

- **Modo Creación:** Si se ingresa directamente a *crearEntrada.html*, el título muestra "Crear Nueva Entrada" y el botón de submit indica "Guardar Registro". Todos los campos se muestran vacíos.
- **Modo Edición:** Si la URL contiene un parámetro único (ej. ``crearEntrada.html?id=1779261936299``), el script asíncrono carga los datos correspondientes en el cliente, pobla automáticamente todos los inputs, resuelve los IDs de etiquetas y categorías en texto legible y cambia los textos a "Modificar Asociación" y "Guardar Cambios".
- **Campos Dinámicos Especiales:**
 - **Categoría Inline:** El selector de categorías incluye una opción especial + *Crear nueva categoría...*. Si se selecciona, JavaScript despliega dinámicamente un campo de texto adicional `#nueva_categoria_input` para definir el nombre de la nueva categoría, el cual se registrará primero en el servidor de forma transparente al guardar.
 - **Etiquetas Múltiples de Texto Libre:** Un input de texto donde el administrador ingresa las etiquetas deseadas separadas por comas. El datalist se autocompleta con hashtags existentes en el servidor. Al guardar, el script divide, válida y crea automáticamente en la base de datos aquellas etiquetas que no existían de forma automática.
 - **Contactos Dinámicos:** Permite agregar múltiples métodos de contacto dinámicos mediante un botón + *Añadir Contacto* que inserta filas con selectores y su valor. Cada fila incluye un botón X de eliminación unitaria.

5. Interfaz de Solicitud de Inclusión (*contacto.html*)

Página de uso público y estructurada que permite a asociaciones no registradas rellenar una propuesta formal para formar parte de la galería pública:

- **Paso 1: Datos de la Asociación:** Formulario de recogida del Nombre de la asociación, Siglas, Descripción del propósito y actividades, Categoría a sugerir, Dirección web/redes y Geolocalización estructurada; País con

autocompletado, Comunidad con datalist predefinido, Provincia obligatoria y Municipio opcional.

- **Paso 2: Datos del Representante y Protección de Datos:** Formulario de información del Representante; Nombre y Apellidos, Cargo dentro de la entidad, Correo electrónico obligatorio y Teléfono opcional. Adicionalmente, cuenta con un aviso legal explícito de protección de datos con una casilla de selección interactiva obligatoria, *#sol_acepta_lopd*.
- **Honeypot Anti-Spam:** Incorpora un campo invisible anti-spam, *#honeypot*, invisible al ojo del usuario común pero rastreable por bots automáticos; si el campo recibe contenido, el script de JavaScript aborta inmediatamente el envío para proteger al servidor contra inundación de peticiones basura.
- **Paso 3: Confirmación de Envío:** Pantalla de éxito oculta por defecto, *#confirmacion-envio*, que sustituye dinámicamente el formulario completo al recibir una respuesta de red satisfactoria del backend. Renderiza un mensaje de felicitación e inyecta la referencia única del registro del servidor.

6. Interfaz y Herramientas Administrativas Avanzadas

Módulos de administración accesibles de manera exclusiva para usuarios autenticados:

- **Herramienta de Carga Masiva (CSV):** Sección de importación ubicada en el panel de control. Permite arrastrar o seleccionar un archivo en formato .csv para su procesamiento masivo. Ofrece una vista con el progreso y reporte de la cantidad de registros importados exitosamente, mapeando las columnas Nombre, Siglas, Descripción, Categoría y Cartera de Servicios de forma automática.
- **Subida y Almacenamiento de Logotipos:** Componente interactivo de carga de archivos, `FileInput` en el formulario de asociación. Permite al administrador cargar archivos de imagen locales. Muestra una barra de estado o previsualización del logo, comunicándose de fondo con */api/upload-logo* y recuperando la URL del servidor física.
- **Personalización Visual del Portal:** Panel de configuración global de apariencia que permite cambiar y guardar en tiempo real estilos generales del portal, tales como el color/estilo de fondo de la interfaz pública, persistiendo la preferencia en *config.json*.

4.7. SEGURIDAD Y CONTROL DE ACCESO

Debido a que la aplicación web gestiona información de asociaciones de salud ligadas al hospital y ofrece un panel de administración para manipular los datos, la seguridad es un pilar fundamental en el desarrollo del sistema. A continuación se detallan las medidas de protección implementadas tanto en el servidor como

en el cliente para garantizar la confidencialidad, integridad y disponibilidad del portal.

4.7.1. Autenticación y Autorización basada en JWT (JSON Web Tokens)

Para la gestión de sesiones de administración se ha implementado un esquema de autenticación desacoplado y stateless utilizando JWT. Este mecanismo evita la necesidad de almacenar sesiones en el servidor, simplificando la arquitectura y reduciendo la superficie de ataque. A continuación se describe en profundidad cada componente del sistema.

4.7.1.1. Configuración de Seguridad en el Servidor

El módulo *server.py* se definen tres constantes críticas de seguridad que gobiernan todo el ciclo de vida de la autenticación:

Constante	Origen	Valor por defecto	Propósito
SECRET_KEY	Variable de entorno	Cadena en desarrollo (Cambiar en producción)	Clave criptográfica utilizada para firmar y verificar la integridad de los tokens
ADMIN_USERNAME	Variable de entorno	admin	nombre del administrador contra el que se validan credenciales
ADMIN_PASSWORD	Variable de entorno	1234	Contraseña del administrador, externalizada mediante variables de entorno para no insertar datos en el código fuente
ALGORITHM		HS256	Algoritmo simétrico utilizado para la firma criptográfica

```

19 # Configuración de Seguridad
20 SECRET_KEY = os.environ.get("SECRET_KEY", "cambia_esto_por_una_clave_larga_y_secreta_en_produccion")
21 ADMIN_USERNAME = os.environ.get("ADMIN_USERNAME", "admin")
22 ADMIN_PASSWORD = os.environ.get("ADMIN_PASSWORD", "1234")
23 ALGORITHM = "HS256"

```

Adicionalmente, se declara un esquema de autenticación OAuth2 de tipo Password Bearer que instruye a FastAPI sobre dónde buscar el token en las peticiones entrantes:

```

47 # Esquema de autenticación (indica dónde buscar el token)
48 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
49

```

Este esquema espera que el token se transmita en la cabecera HTTP *Authorization* con el prefijo Bearer, siguiendo el estándar RFC 6750.

4.7.1.2. Flujo Completo de Autenticación

El ciclo de autenticación sigue un flujo de 4 fases bien definidas:

Fase 1 — Envío de Credenciales (Cliente → Servidor)

El administrador introduce su usuario y contraseña en el formulario de login (*login.html*). El script *login.js* serializa las credenciales en formato *application/x-www-form-urlencoded* y las envía mediante una petición *POST* asíncrona al endpoint */token*:

```

17 const formData = new URLSearchParams();
18 formData.append('username', usuario);
19 formData.append('password', password);
20
21 try {
22   const response = await fetch('/token', {
23     method: 'POST',
24     headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
25     body: formData
26   });
27

```


Fase 2 — Validación de Identidad y Generación del Token (Servidor)

El endpoint `POST /token` recibe las credenciales a través del objeto `OAuth2PasswordRequestForm` inyectado automáticamente por FastAPI. El servidor compara las credenciales recibidas contra las variables de entorno seguras:

```
145 @app.post("/token")
146 async def login(form_data: OAuth2PasswordRequestForm = Depends()):
147     # Validación segura contra credenciales definidas
148     if form_data.username == ADMIN_USERNAME and form_data.password == ADMIN_PASSWORD:
149         access_token = create_access_token(data={"sub": form_data.username})
150         return {"access_token": access_token, "token_type": "bearer"}
151     raise HTTPException(
152         status_code=status.HTTP_401_UNAUTHORIZED,
153         detail="Credenciales incorrectas",
154         headers={"WWW-Authenticate": "Bearer"},
155     )
156
```

Si las credenciales son válidas, se invoca la función `create_access_token()`, que construye el payload del token con dos campos:

- **sub**: Identifica al usuario autenticado, mediante el nombre de usuario.
- **exp**: Marca temporal UTC que define el momento exacto en que el token dejará de ser válido (30 minutos desde su creación).

```
127 def create_access_token(data: dict):
128     to_encode = data.copy()
129     # Expiración real de 30 minutos para mitigar secuestro de sesión
130     expire = datetime.utcnow() + timedelta(minutes=30)
131     to_encode.update({"exp": expire})
132     return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
133
```

La función `jwt.encode()` de la librería PyJWT firma criptográficamente el payload completo `{"sub": "admin", "exp": <timestamp>}` utilizando la `SECRET_KEY` y el algoritmo `HS256`, produciendo una cadena codificada en Base64URL compuesta por tres segmentos separados por puntos: `header.payload.signature`.

Fase 3 — Almacenamiento del Token (Cliente)

Cuando el servidor responde con el token, el cliente lo almacena de forma persistente en el *localStorage* del navegador bajo la clave *adminToken*, y redirige al panel de administración:

```
30     const data = await response.json();
31     localStorage.setItem('adminToken', data.access_token);
32     window.location.href = 'index.html';
```

(línea 30-32 del script login.js)

Fase 4 — Inyección Automática en Peticiones Protegidas (Cliente → Servidor)

A partir de este momento, todas las operaciones de escritura, modificación o eliminación que realice el administrador requieren la inclusión del token en la cabecera HTTP. El script *crearEntrada.js* centraliza esta lógica en una función auxiliar *apiRequest()* que inyecta automáticamente la cabecera *Authorization: Bearer <token>* en cada petición:

```
43     async function apiRequest(url, method = 'GET', body = null) {
44         const headers = {
45             'Authorization': `Bearer ${token}`
46         };
47         if (body) {
48             headers['Content-Type'] = 'application/json';
49         }
50
51         const config = {
52             method,
53             headers
54         };
55         if (body) {
56             config.body = JSON.stringify(body);
57         }
58
59         const response = await fetch(url, config);
60
61         if (response.status === 401) {
62             alert("Tu sesión ha expirado.");
63             localStorage.removeItem('adminToken');
64             window.location.href = 'login.html';
65             return null;
66         }
67
68         if (!response.ok) {
69             let errorMsg = 'Error en la petición de API';
70             try {
71                 const err = await response.json();
72                 if (err.detail) {
73                     if (typeof err.detail === 'string') {
74                         errorMsg = err.detail;
75                     } else if (Array.isArray(err.detail)) {
76                         errorMsg = err.detail.map(e => `${e.loc.join('.')}: ${e.msg}`).join('\n');
77                     } else {
78                         errorMsg = JSON.stringify(err.detail);
79                     }
80                 }
81             } catch (e) {
82                 // No es JSON o no tiene detail
83             }
84             throw new Error(errorMsg);
85         }
86
87         return await response.json();
88     }
```

(línea 43-88 del script crearEntrada.js)

4.7.1.4. Cobertura de Protección: Endpoints Protegidos por JWT

La siguiente tabla enumera exhaustivamente todos los endpoints del servidor que requieren autenticación JWT mediante la dependencia *Depends(verify_token)*:

Método HTTP	Endpoint	Función del servidor	Acción protegida
POST	/api/config	guardar_config()	Guardar configuración visual global
GET	/api/solicitudes	listar_solicitudes()	Listar todas las solicitudes de inclusión recibidas
PUT	/api/solicitudes/{id}	actualizar_estado_solicitud()	Cambiar el estado de una solicitud
DELETE	/api/solicitudes/{id}	eliminar_solicitud()	Eliminar permanentemente una solicitud del registro
POST	/api/upload-logo	subir_logo()	Subir una imagen de logotipo de asociación al servidor
POST	/api/importar	importar_asociaciones()	Importar asociaciones masivamente desde un archivo CSV
POST	/api/{recurso}	escribir_datos()	Crear nuevos registros de asociaciones, categorías o etiquetas
PUT	/api/{recurso}/{id}	actualizar_registro()	Modificar un registro existente de cualquier recurso
DELETE	/api/{recurso}/{id}	eliminar_registro()	Eliminar un registro de cualquier recurso

Los endpoints de lectura pública (*GET /api/{recurso}*, *GET /api/config*, *POST /api/solicitudes*) permanecen sin protección intencionadamente, ya que sirven el catálogo público de asociaciones y permiten el envío de solicitudes por parte de usuarios anónimos.

4.7.1.5. Gestión de Sesión y Control de Acceso en el Cliente.

La capa del cliente implementa mecanismos complementarios de control de acceso:

- **Guardián de Rutas (*auth.js*):** Un script cargado en todas las páginas de administración verifica al inicio la presencia del token en *localStorage*. Si no existe y la página actual no es *login.html* ni una página pública, redirige automáticamente al formulario de login, impidiendo el acceso no autenticado a las vistas protegidas.

```
2     const token = localStorage.getItem('adminToken');
3     const path = window.location.pathname;
4
5     const isLoginPage = path.endsWith('login.html');
6     const isPublicPage = path.endsWith('index.html') || path === '/' || path.endsWith('/');
7
8     if (!token && !isLoginPage && !isPublicPage) {
9         window.location.href = 'login.html';
10        return;
11    }
```

- **Visibilidad Condicional de Elementos:** El mismo script muestra u oculta selectivamente elementos del DOM mediante las clases CSS *.req-admin* y *.req-publico*, garantizando que los botones de administración (editar, eliminar, importar) solo sean visibles cuando existe una sesión activa.

```
13     if (token) {
14         document.querySelectorAll('.req-admin').forEach(el => el.style.display = 'block');
15         document.querySelectorAll('.req-publico').forEach(el => el.style.display = 'none');
16     } else {
17         document.querySelectorAll('.req-admin').forEach(el => el.style.display = 'none');
18         document.querySelectorAll('.req-publico').forEach(el => el.style.display = 'block');
19     }
20 }
```

- **Cierre de Sesión Explícito (Logout):** El botón de logout (`#btn-logout`) elimina el token del `localStorage` y redirige a la página principal, destruyendo la sesión del navegador sin necesidad de comunicarse con el servidor.

```
21     const btnLogout = document.getElementById('btn-logout');
22     if (btnLogout) {
23         btnLogout.addEventListener('click', () => {
24             localStorage.removeItem('adminToken');
25             window.location.href = 'index.html';
26         });
27     }
28 }
```

- **Detección Proactiva de Expiración:** Cuando cualquier petición protegida recibe una respuesta HTTP 401, el cliente asume que el token ha expirado, limpia el `localStorage` y redirige al login, proporcionando una experiencia fluida de autenticación

4.7.2. Inyección de Cabeceras

Para proteger al portal contra ataques comunes a nivel de navegador y red, el servidor FastAPI integra un middleware de interceptación que inyecta automáticamente cabeceras HTTP de seguridad robustas en cada respuesta emitida por el servidor, independientemente del endpoint solicitado. A continuación se describe en profundidad la arquitectura del middleware y el papel defensivo de cada cabecera.

4.7.2.1. Arquitectura del Middleware Interceptor

El middleware se implementa como una clase Python que hereda de `BaseHTTPMiddleware` del framework Starlette (capa subyacente de FastAPI). Su funcionamiento sigue el patrón de interceptor de cadena (chain-of-responsibility):

1. La petición HTTP entrante llega al middleware antes de alcanzar cualquier endpoint de la API.
2. El middleware delega la petición al siguiente eslabón de la cadena (`call_next(request)`), que eventualmente ejecuta la lógica del endpoint correspondiente.
3. Una vez generada la respuesta, el middleware la intercepta antes de enviarla al cliente e inyecta las cabeceras de seguridad en ella.
4. La respuesta enriquecida se devuelve al navegador.

```

27 # Middleware para inyectar Cabeceras de Seguridad y prevenir ataques (Clickjacking, XSS, MIME Sniffing)
28 class SecurityHeadersMiddleware(BaseHTTPMiddleware):
29     async def dispatch(self, request, call_next):
30         response: Response = await call_next(request)
31         response.headers["X-Frame-Options"] = "DENY"
32         response.headers["X-Content-Type-Options"] = "nosniff"
33         response.headers["X-XSS-Protection"] = "1; mode=block"
34         response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
35         response.headers["Content-Security-Policy"] = (
36             "default-src 'self'; "
37             "script-src 'self' 'unsafe-inline'; "
38             "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; "
39             "font-src 'self' https://fonts.gstatic.com; "
40             "img-src 'self' data: http: https; "
41             "frame-src https://www.youtube.com https://youtube.com;"
42         )
43         return response
44
45 app.add_middleware(SecurityHeadersMiddleware)
46

```

Este enfoque garantiza que ninguna respuesta del servidor escape sin las cabeceras de protección, incluyendo las respuestas a archivos estáticos, endpoints de la API REST, y redirecciones.

4.7.2.2. Resumen de Cabeceras Inyectadas

Cabecera HTTP	Valor Configurado	Ataque Mitigado	Estándar
<i>X-Frame-Options</i>	<i>DENY</i>	Clickjacking	RFC 7034
<i>Content-Security-Policy</i>	Política multi-directiva (ver 4.7.2.4)	XSS, inyección de recursos, exfiltración de datos	W3C CSP Level 2
<i>X-Content-Type-Options</i>	<i>nosniff</i>	MIME Sniffing	Fetch Standard
<i>X-XSS-Protection</i>	<i>1; mode=block</i>	XSS reflejado	No estandarizado
<i>Referrer-Policy</i>	<i>strict-origin-when-cross-origin</i>	Filtración de URLs internas	W3C Referrer Policy

4.7.2.3. Protección contra Clickjacking

Ataque Mitigado: El clickjacking (o UI redressing) consiste en que un atacante embebe el portal legítimo dentro de un `<iframe>` transparente superpuesto en su propia página maliciosa. El administrador, creyendo interactuar con la página del atacante, en realidad realiza clics sobre los botones de la aplicación real (eliminar registros, modificar configuración, etc.) sin ser consciente de ello.

Mecanismo de Defensa: La cabecera *X-Frame-Options: DENY* instruye al navegador para que bloquee por completo la carga del portal dentro de cualquier *<iframe>*, *<frame>*, *<embed>* u *<object>*, sin importar el origen del sitio que intente embebido.

Alcance: Se ha elegido el valor DENY (el más restrictivo) en lugar de SAMEORIGIN porque la aplicación no requiere ser embebida ni siquiera desde su propio dominio. Esto elimina por completo el riesgo de clickjacking.

4.7.2.4. Política de Seguridad de Contenido

La cabecera *Content-Security-Policy* es la capa de defensa más exhaustiva del middleware. **Define una lista blanca de orígenes** autorizados para cada tipo de recurso que el navegador puede cargar, bloqueando cualquier recurso externo no listado explícitamente. Esto mitiga de raíz los ataques de inyección de código, exfiltración de datos y carga de recursos maliciosos.

La política se compone de 6 directivas configuradas de forma granular para satisfacer las necesidades funcionales del portal sin relajar la seguridad:

Directiva default-src 'self'

Establece la política por defecto para todos los tipos de recursos no cubiertos explícitamente por otras directivas. De manera que solo se permite la carga de recursos (scripts, estilos, fuentes, conexiones AJAX, etc.) desde el propio origen del servidor. Cualquier recurso de un dominio externo no listado en las directivas específicas será bloqueado automáticamente.

Directiva script-src 'self' 'unsafe-inline'

Controla los orígenes desde los que se pueden ejecutar scripts de JavaScript. **'self'** permite la carga de archivos .js servidos desde el propio servidor (los scripts de /html/js/). Mientras que **'unsafe-inline'**, permite la ejecución de scripts inline (*<script>...</script>* embebidos directamente en el HTML). Esta directiva es necesaria porque la arquitectura Vanilla JS del portal emplea inicializaciones inline en algunos archivos HTML. Se compensa este riesgo con la sanitización XSS en el cliente (documentada en la sección 4.7.3).

Directiva style-src 'self' 'unsafe-inline' https://fonts.googleapis.com

Controla los orígenes autorizados para hojas de estilo CSS, **'self'**; permite las hojas de estilo locales (ej. /html/css/style_Form.css), **'unsafe-inline'**, permite estilos inline (*style="..."*) aplicados dinámicamente por JavaScript para funcionalidades como la

previsualización del fondo global, las tarjetas de galería, o la visibilidad condicional de elementos administrativos.

Mientras que **<https://fonts.googleapis.com>**, autoriza la carga de la hoja de estilos de Google Fonts. El portal importa las familias tipográficas Montserrat y Outfit desde este CDN, como se observa en la directiva `@import` de `style_Form.css`:

```
html > css > {} style_Form.css > ...
1 @import url('https://fonts.googleapis.com/css2?family=Montserrat:wght@400;500;600;700&family=Outfit:wght@300;400;500;600;700&display=swap');
2
```

Directiva `font-src 'self'` **<https://fonts.gstatic.com>**

Controla los orígenes desde los que el navegador puede descargar archivos de fuentes tipográficas, **<https://fonts.gstatic.com>**, es el CDN de Google que alojan físicamente los archivos binarios de las fuentes. Mientras que **fonts.googleapis.com** sirve la hoja de estilos CSS con las declaraciones `@font-face`, los archivos de fuentes reales se descargan desde **fonts.gstatic.com**. Ambos dominios son necesarios para que las fuentes Montserrat y Outfit se carguen correctamente.

Directiva `img-src 'self' data: http: https:`

Controla los orígenes autorizados para la carga de imágenes.

'self': Permite imágenes locales servidas desde el propio servidor (fondos institucionales en `/html/img/`, logotipos subidos en `/html/img/logos/`).

data:: Autoriza imágenes embebidas directamente en el HTML como cadenas Base64 (`data:image/png;base64,...`). Esto es necesario para la funcionalidad de previsualización instantánea del logo en el formulario de creación de entradas (`crearEntrada.js`), donde se usa un `FileReader` para mostrar la imagen seleccionada antes de subirla al servidor.

http: y https:: Permiten la carga de logotipos de asociaciones desde URLs externas públicas (por ejemplo, el logo oficial de una asociación alojado en su propio sitio web), ya que el campo logo del modelo de datos acepta tanto rutas locales del servidor como URLs externas completas.

Directiva `frame-src` **<https://www.youtube.com>** **<https://youtube.com>**

Controla los orígenes que pueden cargarse dentro de elementos `<iframe>` en las páginas del portal. La ficha de detalle de cada asociación (`detalle.js`) incluye una galería multimedia de vídeos que renderiza reproducciones embebidas de YouTube mediante iframes generados dinámicamente:

```
231 const iframe = document.createElement('iframe');
232 iframe.src = 'https://www.youtube.com/embed/${videoId}';
233 iframe.title = 'Video de YouTube';
234 iframe.frameBorder = '0';
235 iframe.allow = 'accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture; web-share';
236 iframe.allowFullscreen = true;
237 iframe.loading = 'lazy';
```

(`detalle.js`, líneas 231-237)

La directiva autoriza exclusivamente los dominios `www.youtube.com` y `youtube.com` como orígenes válidos para iframes. Cualquier intento de cargar un iframe desde otro dominio (por ejemplo, un reproductor de vídeo malicioso) será bloqueado por el navegador, impidiendo ataques de inyección de contenido multimedia no confiable

4.7.2.5. Prevención de MIME Sniffing

Vector de Ataque Mitigado: El MIME sniffing es un comportamiento de ciertos navegadores que intentan "adivinar" el tipo real de un recurso ignorando la cabecera `Content-Type` declarada por el servidor. Un atacante podría explotar esto subiendo un archivo con extensión `.jpg` que internamente contiene código JavaScript. Sin esta protección, el navegador podría detectar el contenido como script y ejecutarlo.

Mecanismo de Defensa: La cabecera `X-Content-Type-Options: nosniff` instruye al navegador para que respete estrictamente el tipo MIME declarado por el servidor en cada respuesta, sin intentar inferir un tipo alternativo.

Esta cabecera es especialmente crítica porque el portal permite la subida de archivos de imagen y sirve archivos estáticos desde el directorio `/html/`. Sin `nosniff`, un atacante podría intentar camuflar código ejecutable como una imagen y explotar el MIME sniffing para lograr ejecución de código en el navegador de otro usuario o del administrador

4.7.2.6. Filtro de XSS Reflejado

El XSS reflejado (Cross-Site Scripting reflejado) ocurre cuando un atacante inyecta código malicioso en los parámetros de una URL y la aplicación refleja ese código de vuelta en la respuesta HTML sin sanitizar. Si la víctima hace clic en la URL manipulada, el código se ejecuta en su navegador.

La cabecera activa el filtro XSS nativo de ciertos navegadores (principalmente Internet Explorer y versiones legacy de Chrome). El valor `mode=block` indica que, si el filtro detecta un intento de XSS reflejado, debe bloquear completamente la carga de la página en lugar de intentar sintetizar el contenido.

Aunque los navegadores modernos han deprecado esta cabecera en favor de políticas CSP robustas, se mantiene como defensa para proteger a usuarios que accedan con navegadores antiguos o sin soporte completo de CSP.

4.7.2.7. Política de Privacidad de Referencias

Cuando un usuario navega desde el portal hacia un enlace externo, el navegador incluye por defecto una cabecera Referer que revela la URL completa de la página de origen. Esto podría exponer rutas administrativas internas, IDs de sesión en la URL, o la estructura de navegación del panel de administración.

La política *strict-origin-when-cross-origin* establece reglas diferenciadas según el destino:

Escenario de Navegación	Información Compartida
Mismo origen	Funcionamiento normal de navegación interna.
Origen cruzado con mismo protocolo	sin incluir el path ni los parámetros de la URL.
Degradación de protocolo	Se suprime completamente la cabecera Referer para evitar filtración de datos en canales no cifrados

El portal contiene numerosos enlaces de contacto hacia sitios externos de las asociaciones (redes sociales, páginas web oficiales). Sin esta política, cada clic en un contacto externo revelaría la URL completa de la ficha de detalle, incluyendo el ID de la asociación, al sitio de destino. Con la política configurada, solo se comparte el dominio del portal, preservando la privacidad de la navegación interna.

4.7.3. Sanitización y Prevención de Scripts entre Sitios (XSS) en el Cliente

En el entorno del navegador, se han adoptado políticas y utilidades preventivas estrictas en los scripts de JavaScript para erradicar vulnerabilidades de inyección de código. Las medidas cubren los tres vectores clásicos de XSS: almacenado (datos maliciosos persistidos en el servidor y renderizados a otros usuarios), reflejado (código inyectado en parámetros de URL) y basado en DOM (manipulación directa del árbol de elementos HTML). A continuación se describe cada técnica implementada.

4.7.3.1. Manipulación Segura del DOM

Si los datos textuales procedentes de la API se insertan en el DOM usando la propiedad *innerHTML*, un atacante podría almacenar una cadena maliciosa como nombre de asociación (por ejemplo, `<img src=x onerror="alert(document.cookie)">`) que se ejecutaría en el navegador de cada visitante que acceda a la ficha.

En todas las vistas públicas y administrativas se ha establecido como norma de desarrollo el uso exclusivo de la propiedad *textContent* (o *innerText*) para insertar datos textuales procedentes del servidor. Esta propiedad trata todo el contenido como texto plano, escapando automáticamente cualquier carácter HTML especial (<, >, &, ", ') e impidiendo que se interprete como marcado ejecutable.

Renderizado de la ficha de detalle de una asociación

```
114 // Textos directos seguros
115 detalleNombre.textContent = asoc.nombre_asociacion;
116 detalleSiglas.textContent = asoc.siglas;
117 detalleDesc.textContent = asoc.descripcion;
118 detalleServicios.textContent = asoc.cartera_servicios;
119
120 // Resolver Categoría
121 const catObj = categorias.find(c => c.id === asoc.categoria);
122 detalleCategoria.textContent = catObj ? catObj.nombre : 'No especificada';
123
```

Renderizado de tarjetas en el catálogo público

```
196 const h3 = document.createElement('h3');
197 h3.textContent = asoc.nombre_asociacion;
198
199 const divSiglas = document.createElement('div');
200 divSiglas.className = 'insignia-filtro';
201 divSiglas.textContent = asoc.siglas;
202
203 // Contenedor de Etiquetas Múltiples en la tarjeta
204 const tagsDiv = document.createElement('div');
205 tagsDiv.className = 'contenedor-tags-tarjeta';
206 if (asoc.etiquetas && asoc.etiquetas.length > 0) {
207   asoc.etiquetas.forEach(etiId => {
208     const etiObj = etiquetas.find(e => e.id === etiId);
209     if (etiObj) {
210       const tagSpan = document.createElement('span');
211       tagSpan.className = 'badge-tag-tarjeta';
212       tagSpan.textContent = etiObj.nombre;
213       tagsDiv.appendChild(tagSpan);
214     }
215   });
216 }
217
```

Todos los datos dinámicos procedentes de la API que representan texto informativo (nombres, siglas, descripciones, categorías, etiquetas, mensajes de error, estados de solicitudes) se insertan con `textContent`. El uso de `innerHTML` se reserva exclusivamente para el marcado estático controlado por el desarrollador, nunca para datos introducidos por el usuario.

4.7.3.2. Sanitización de URLs

Un atacante podría registrar una asociación con una URL de logo o contacto que use el pseudoprotocolo `javascript:` (`javascript:alert(document.cookie)`). Cuando el navegador renderiza un `` o un `<a href="javascript:...">`, el código malicioso se ejecutaría en el contexto de la sesión del usuario víctima.

Se ha implementado una función validadora `safeUrl(url)` replicada en los dos scripts de renderizado público (`index.js` y `detalle.js`) que analiza cada URL antes de asignarla a un atributo `src` o `href`.

`index.js`

```
47
48 // --- UTILERÍAS DE SEGURIDAD (MITIGACIÓN XSS) ---
49 function safeUrl(url) {
50   if (!url) return '';
51   const cleanUrl = url.trim();
52   if (cleanUrl.toLowerCase().startsWith('javascript:')) {
53     console.warn('URL bloqueada por seguridad (intento de inyección javascript:)', cleanUrl);
54     return '';
55   }
56   return cleanUrl;
57 }
58
```

`detalle.js`

```
62
63 // --- UTILERÍAS DE SEGURIDAD (MITIGACIÓN XSS) ---
64 function safeUrl(url) {
65   if (!url) return '';
66   const cleanUrl = url.trim();
67   if (cleanUrl.toLowerCase().startsWith('javascript:')) {
68     console.warn('URL bloqueada por seguridad (intento de inyección javascript:)', cleanUrl);
69     return '';
70   }
71   return cleanUrl;
72 }
```

Proceso de validación:

1. Control de nulidad: Si la URL está vacía, `null` o `undefined`, devuelve una cadena vacía segura.
2. Eliminación de espacios: `trim()` elimina espacios en blanco al inicio y al final, previniendo evasiones como `javascript:alert(1)`.
3. **Detección de protocolo malicioso:** La comprobación `toLowerCase().startsWith('javascript:')` detecta el pseudoprotocolo `javascript:` con

independencia de mayúsculas/minúsculas (*JavaScript*;; *JAVASCRIPT*;; *jAvAsCrIpT*;; etc.).

4. Bloqueo y registro: Si se detecta un intento, la URL se neutraliza devolviendo una cadena vacía y se emite un aviso en la consola para depuración/auditoría.

Archivo	Línea	Uso
index.js	190	<i>img.src = safeUrl(asoc.logo)</i> — Carga de logotipos en las tarjetas del catálogo
detalle.js	108	<i>detalleLogo.src = safeUrl(asoc.logo)</i> — Carga del logo en la ficha de detalle
detalle.js	193	<i>a.href = safeUrl(cleanVal)</i> — Renderizado de URLs de contacto

4.7.3.3. Escapado de HTML en Atributos

En los formularios de administración, cuando se precargan los datos existentes de una asociación para editarla, los valores del servidor se insertan en atributos value de inputs HTML mediante template literals. Si un dato almacenado contuviera caracteres especiales HTML (por ejemplo, un nombre de contacto como "<script>alert(1)</script>"), el atacante podría romper el atributo e inyectar código ejecutable.

Se han implementado dos funciones de escapado de caracteres HTML que reemplazan los 5 caracteres peligrosos por sus entidades HTML equivalentes:

```
137
138 // Escapar caracteres para HTML seguro en inputs
139 function escapeHTML(str) {
140     if (!str) return '';
141     return str.replace(/&<>'"/g,
142         tag => ({
143             '&': '&amp;',
144             '<': '&lt;',
145             '>': '&gt;',
146             '"': '&#39;',
147             "'": '&quot;',
148         }[tag] || tag)
149     );
150 }
```

```

1031 function escapeHTMLsol(str) {
1032     if (!str) return '';
1033     return String(str).replace(/<>"/g, tag => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[tag] || tag));
1034 }
1035

```

Tabla de caracteres escapados:

Carácter Original	Entidad HTML	Ataque Prevenido
&	&	Inicio de entidades HTML inesperadas
<	<	Apertura de etiquetas HTML/script
>	>	Cierre de etiquetas HTML
"	"	Ruptura de atributos entre comillas dobles
'	'	Ruptura de atributos entre comillas simples

escapeHTML(): Se aplica al precargar valores en los inputs de los formularios de edición de asociaciones (campos de contacto y vídeos de YouTube):

crearEntrada.js — Precarga segura de contactos en modo edición (líneas 107-108)

```

`<input type="text" name="otro_tipo_contacto[]" value="
${escapeHTML(customValue)}
" ...>
<input type="text" name="valor_contacto[]" value="
${escapeHTML(valor)}
" ...>`

```

[illegible]

Un atacante podría almacenar una dirección de correo electrónico maliciosa que al renderizarse en un enlace *mailto:* inyecte cabeceras SMTP adicionales o código JavaScript. Por ejemplo, *test@evil.com%0D%0ABcc:spam@attacker.com* podría añadir destinatarios ocultos al correo.

```
184 // Enlace o texto con su validación XSS correspondientes
185 const cleanVal = contacto.valor.trim();
186 if (cleanTipo === 'email') {
187     const a = document.createElement('a');
188     a.href = `mailto:${encodeURIComponent(cleanVal)}`;
189     a.textContent = cleanVal;
190     li.appendChild(a);
```